UninaFoodLab

Progettazione e realizzazione del sistema

Relazione di progetto

Daniele De Blasio

Indice

1 Introduzione **1**

2 Analisi dei requisiti e scelte di dominio **1**

2.1 Assunzioni e regole di dominio 1

3 Modello del dominio **2**

3.1 UML concettuale: identità, corsi e adesioni 2

3.2 UML concettuale: ricettario, menu e copie 3

3.3 UML concettuale: comunicazioni e lettura 4

3.4 Modello entità-relazione 4

4 Ristrutturazione e schema relazionale **8**

4.1 UML ristrutturato: account e profili 8

4.2 UML ristrutturato: corsi, iscrizioni e adesioni 9

4.3 UML ristrutturato: ricettario, menu e copie 10

4.4 UML ristrutturato: comunicazioni e destinatari 11

4.5 UML ristrutturato: storico e supporto dello schema 12

4.6 Dizionario delle classi concettuali 13

4.7 Dizionario delle associazioni 13

4.8 Schema logico e dizionario degli attributi 14

4.9 app.account 16

4.10 app.account_password_reset 16

4.11 app.auth_session 17

4.12 app.category 17

4.13 app.chef_invitation 17

4.14 app.chef_profile 18

4.15 app.course 18

4.16 app.course_session 19

4.17 app.enrollment 19

4.18 app.ingredient 20

4.19 app.notification 20

4.20 app.notification_recipient 21

4.21 app.participant_profile 21

4.22 app.practical_attendance 21

4.23 app.recipe 22

4.24 app.recipe_ingredient 22

4.25 app.schema_metadata 22

4.26 app.session_change_history . . . 23
4.27 app.session_menu_snapshot . . . 23
4.28 app.session_outcome . . . 24
4.29 app.session_preparation_revision . . . 24
4.30 app.session_recipe . . . 25
4.31 Dizionario dei vincoli . . . 25

5 Componenti applicativi e classi di design **31**
5.1 Creazione del corso e coordinamento . . . 32
5.2 Valori del corso e calendario . . . 33
5.3 Gestione delle adesioni . . . 34
5.4 Lettura della pianificazione . . . 35
5.5 Accesso, registrazione e gestione dell'account . . . 36
5.6 Ricettario, ingredienti e categorie . . . 37
5.7 Calendario dei corsi e menu delle sessioni . . . 38
5.8 Consultazione e iscrizioni del Partecipante . . . 39
5.9 Comunicazioni e report mensile . . . 40
5.10 Consultazione dei corsi Chef e agenda . . . 41

6 Comportamento di due funzionalità **42**
6.1 Creazione e pubblicazione del corso . . . 42
6.2 Conferma di adesione alla pratica . . . 43

7 Realizzazione della persistenza **44**
7.1 Struttura SQL delle tabelle: casi significativi . . . 44
7.2 Trigger e funzioni SQL: operazioni significative . . . 46
7.3 Autenticazione e gestione delle connessioni . . . 51

1 Introduzione

UninaFoodLab organizza corsi di cucina, sessioni e ingredienti necessari. La relazione descrive la progettazione del sistema e della base di dati, motivando le scelte nei diversi passaggi. Il codice è disponibile nel repository https://github.com/Daniele04-creator/UninaFoodLab.

2 Analisi dei requisiti e scelte di dominio

Lo Chef organizza corsi con sessioni online o pratiche, associa ricette alle pratiche e consulta fabbisogni e report mensili. Il Partecipante si iscrive al corso e aderisce alle singole pratiche: questa distinzione permette di pianificare gli ingredienti in base alle conferme. Le comunicazioni sono consultabili nell'applicazione.

Tabella 1: Analisi dei requisiti e scelte di dominio

ID	Requisito	Interpretazione operativa
R01	Autenticazione e ruoli	Username/password; accessi alle operazioni autorizzati dal token.
R02	Creazione dei corsi	Creazione e pubblicazione atomiche; inizialmente 2–30 sessioni.
R03	Consultazione	Filtri per categoria e stato, ricerca e paginazione.
R04	Programma delle sessioni	Modalità, durata, sede o URL HTTPS, capienza e scadenza delle adesioni.
R05	Menu delle pratiche	Più ricette per sessione e copie delle dosi associate.
R06	Iscrizioni e adesioni	Iscrizione al corso distinta dalla conferma della singola pratica.
R07	Fabbisogni	Dosi della copia moltiplicate per conferme valide; lista della spesa stampabile.
R08	Variazioni e comunicazioni	Programma correggibile e comunicazioni destinate agli iscritti.
R09	Report mensile	Corsi distinti, online/pratiche, media/minimo/massimo delle ricette; grafici.
R10	Persistenza e integrità	Vincoli, trigger e funzioni SQL; migrazioni versionate.

2.1 Assunzioni e regole di dominio

Una sessione non annullata è considerata svolta alla fine prevista, nel fuso Europe/Rome; le ricette del menu sono assunte realizzate. Il report conta corsi distinti e sessioni svolte nel mese dell'appuntamento. Una pratica senza ricette contribuisce con zero; in assenza di pratiche le statistiche sulle ricette sono assenti.

Uno spostamento dell'appuntamento richiede una nuova adesione; la sola correzione di sede, link o scadenza conserva le conferme. Una reiscrizione non riattiva risposte del periodo precedente. I fabbisogni usano le copie del menu e le conferme correnti, senza margine aggiuntivo.

Il calendario consente modifiche alle sole sessioni future e programmate di corsi pubblicati. Una sessione già iniziata o conclusa conserva il proprio appuntamento, anche quando sede e link non vengono modificati.

Le dosi sono riferite a una persona e ammettono al massimo tre decimali senza arrotondamento implicito. I limiti di 2–30 sessioni iniziali e frequenza di 1–31 giorni sono protezioni operative del form di creazione.

3 Modello del dominio

Il modello rappresenta concetti e attributi del dominio. Le molteplicità sono ai due estremi: ciascun valore conta le istanze della classe vicina collegate a una singola istanza opposta. Le classi ripetute mantengono la stessa identità; / indica un valore derivato.

Account e profili sono oggetti distinti, collegati da associazioni. La registrazione crea il profilo richiesto dal ruolo; trigger e routine impediscono profili incompatibili.

3.1 UML concettuale: identità, corsi e adesioni

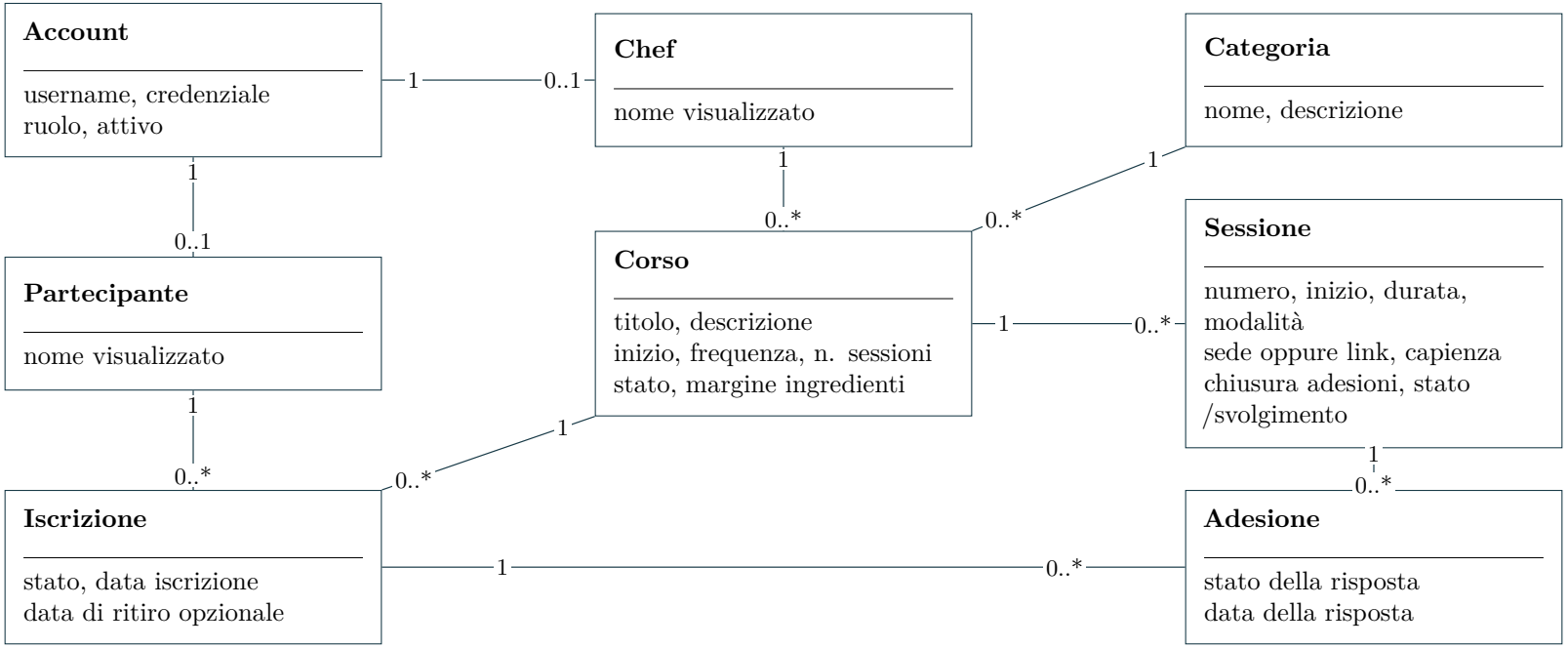


Figura 1: UML concettuale: identità, corsi e adesioni

Iscrizione collega Partecipante e Corso. Adesione collega quella Iscrizione a una Sessione dello stesso corso, distinguendo le risposte dei diversi periodi. La modalità condiziona gli attributi della Sessione; limiti di creazione e controlli tra righe sono nella sezione 4.31.

3.2 UML concettuale: ricettario, menu e copie

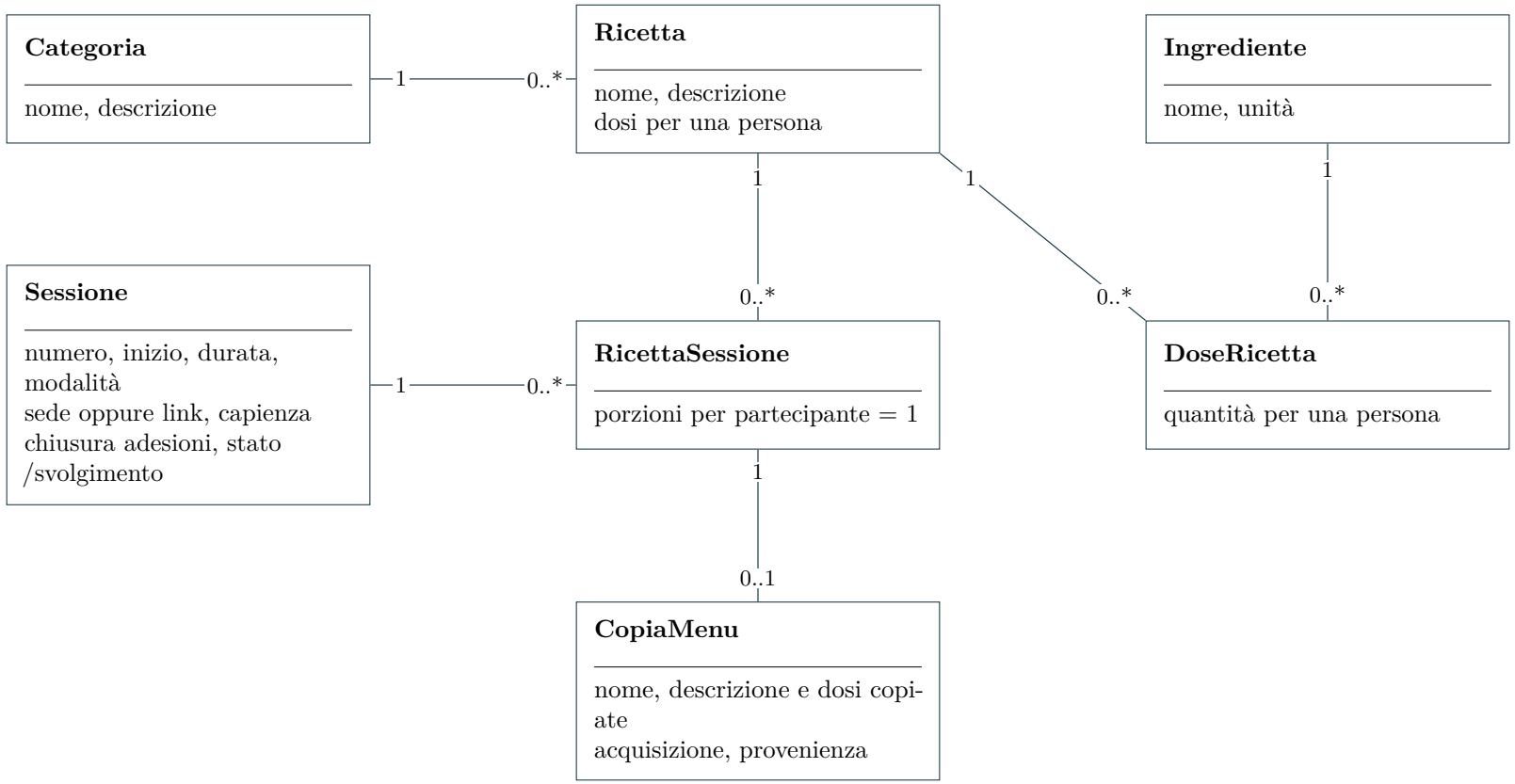


Figura 2: UML concettuale: ricettario, menu e copie

DoseRicetta rende esplicita la quantità del legame tra Ricetta e Ingrediente. RicettaSessione permette più ricette nella stessa pratica; CopiaMenu conserva le dosi e il contesto acquisiti, così le modifiche al ricettario non cambiano una preparazione già associata. Il ricettario è condiviso e non registra un proprietario Chef. La molteplicità 0..1 della copia esprime la struttura; l'associazione ordinaria la crea automaticamente.

3.3 UML concettuale: comunicazioni e lettura

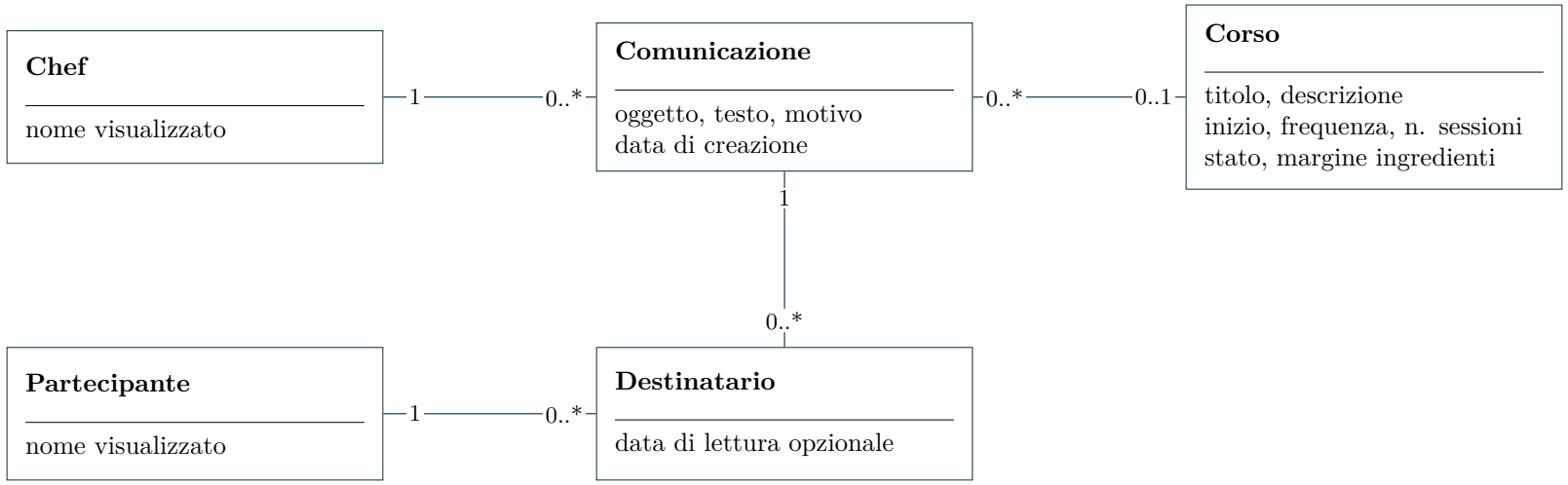


Figura 3: UML concettuale: comunicazioni e lettura

Comunicazione ha sempre un autore Chef. Il Corso è opzionale perché il messaggio può riguardare tutti i corsi dello chef. Destinatario identifica ogni coppia messaggio/partecipante e conserva la lettura senza modificare il contenuto comune. La registrazione del messaggio non rappresenta un invio email o SMS.

3.4 Modello entità-relazione

Il modello ER esplicita gli identificatori e gli attributi. I rettangoli contengono le entità; gli identificatori sono sottolineati. I rombi rappresentano relazioni e possono contenere attributi propri. Le cardinalità minime e massime sono poste vicino all'entità e indicano quante volte ciascuna sua istanza partecipa alla relazione: 0..N ammette anche nessun collegamento.

Iscrizione è rappresentata come entità perché possiede una propria identità ed è richiamata da Adesione. Anche RicettaSessione viene resa esplicita, identificata dalla coppia Sessione/Ricetta, perché la copia del menu dipende da quella coppia. Dose, Adesione e Destinatario rimangono relazioni con attributi; nella ristrutturazione diventano tabelle associative. Le entità ripetute collegano le tre tavole.

3.4.1 Identità, corsi, iscrizioni e adesioni

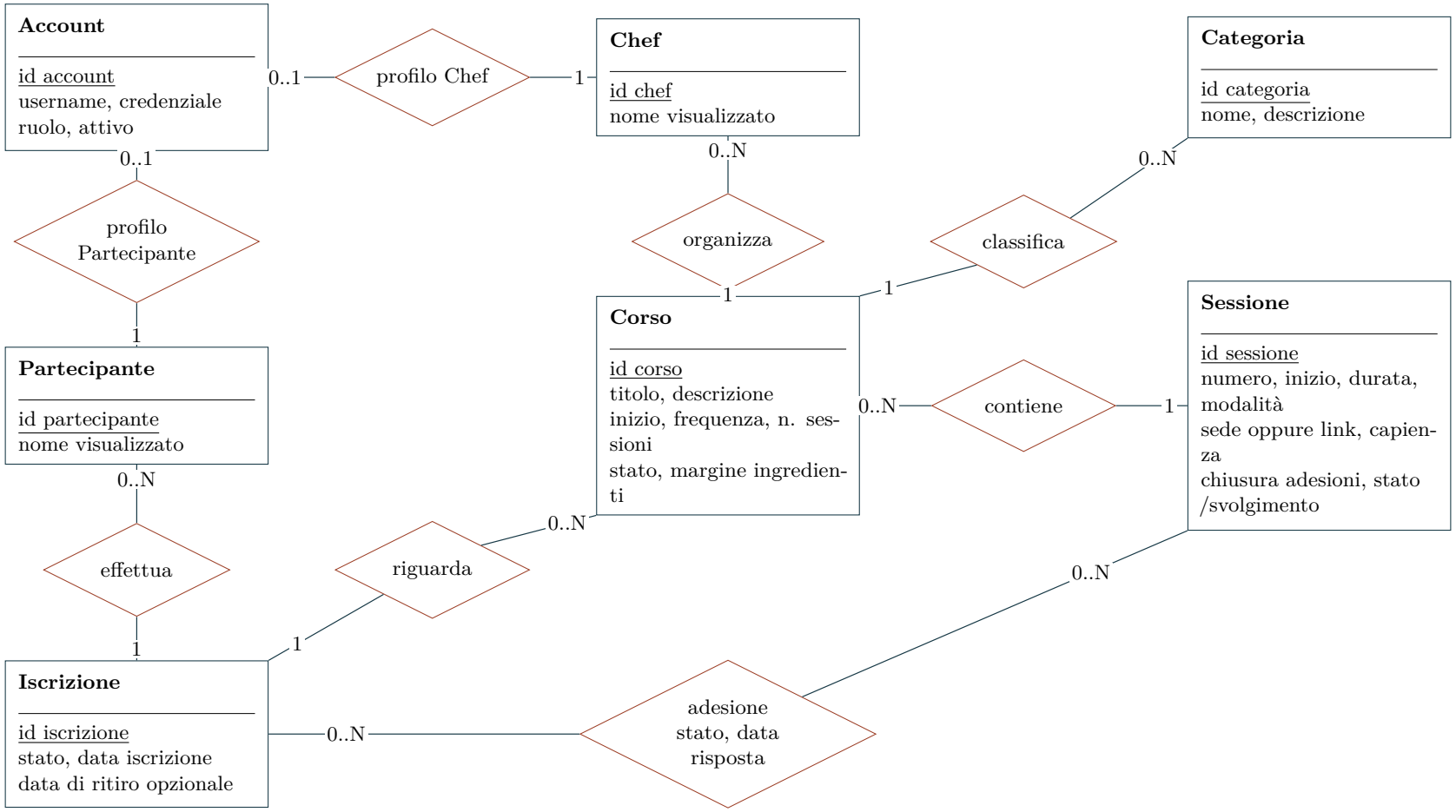


Figura 4: ER: identità, corsi, iscrizioni e adesioni

Iscrizione conserva stato e periodo ed è univoca per Partecipante/Corso. La relazione Adesione è identificata dalla coppia Iscrizione/Sessione; una conferma è ammessa solo per una pratica dello stesso corso. I profili sono alternativi per ruolo. Gli identificatori dei singoli oggetti diventano UUID nello schema logico.

3.4.2 Ricette, ingredienti, menu e copie

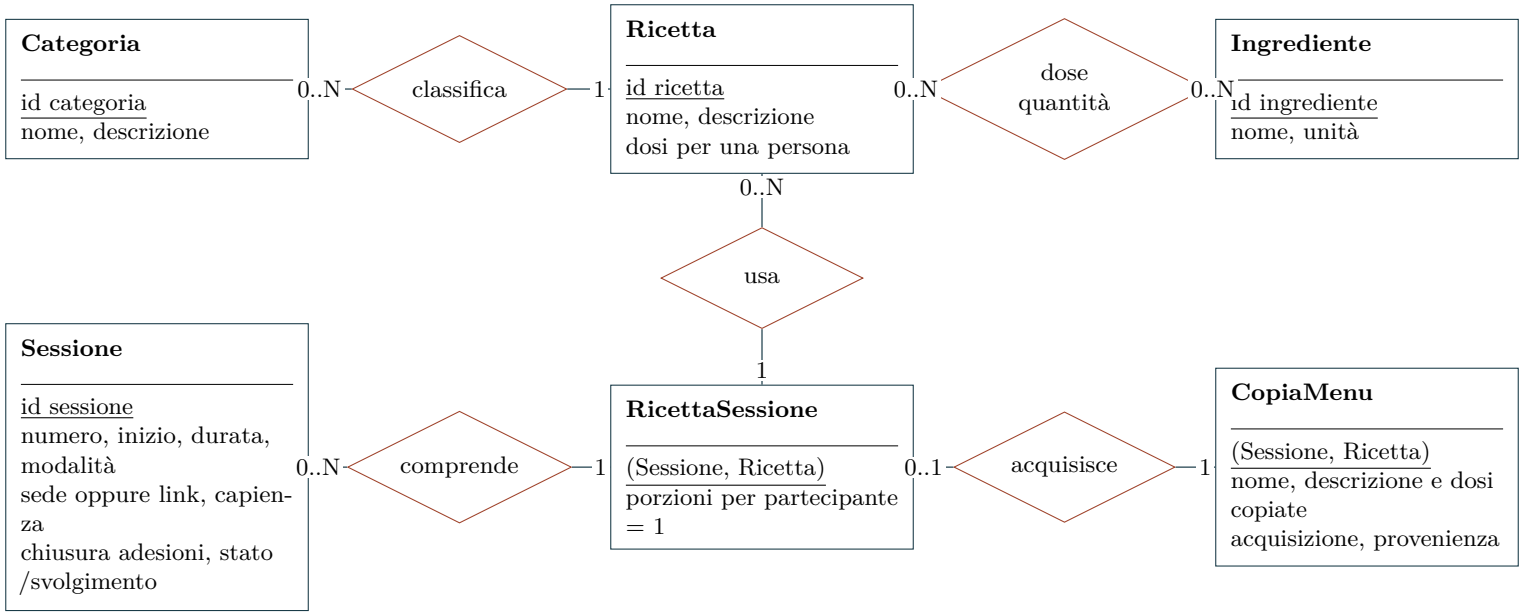


Figura 5: ER: ricette, ingredienti, menu e copie

Dose è identificata dalla coppia Ricetta/Ingrediente e contiene la quantità per una persona. RicettaSessione conserva le porzioni per partecipante; la copia usa lo stesso identificatore composto. Una Sessione online non può avere un menu. Categoria è la stessa entità che classifica i corsi.

3.4.3 Comunicazioni e destinatari

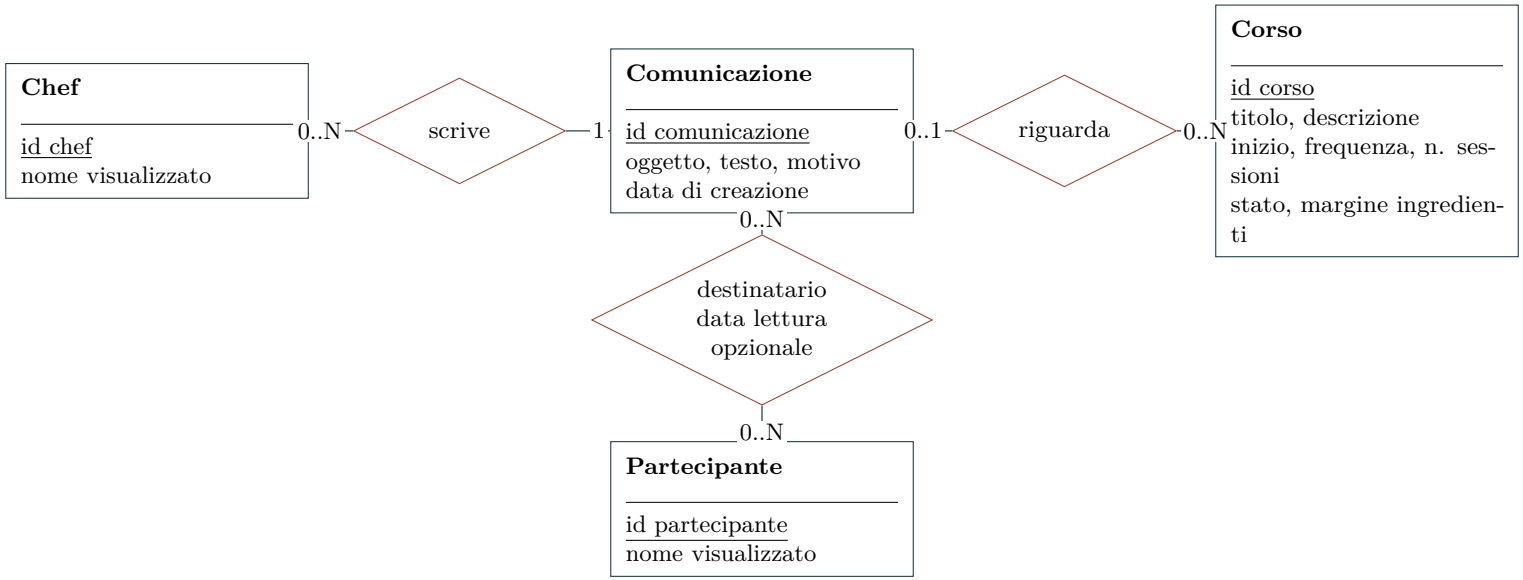


Figura 6: ER: comunicazioni e destinatari

Destinatario è identificato dalla coppia Comunicazione/Partecipante e registra una data di lettura opzionale. Il Corso può essere assente in un messaggio globale; l'autore Chef è sempre presente. La materializzazione dei destinatari conserva a chi è stata indirizzata la comunicazione.

4 Ristrutturazione e schema relazionale

Account e profili diventano tabelle distinte con FK univoca, evitando credenziali duplicate. Le relazioni molti-a-molti diventano associative: Dose, Adesione e Destinatario usano una coppia di FK come PK; Iscrizione mantiene un identificatore autonomo perché viene richiamata dalle adesioni.

Ricette, ingredienti e dosi correnti sono normalizzati. CopiaMenu conserva invece il contesto in JSONB: questa ridondanza impedisce modifiche retroattive della preparazione. La sua FK composta punta all'intera coppia Sessione/Ricetta.

Le cinque tavole comprendono 22 tabelle e 24 FK. I nodi di raccordo sono ripetuti; gli attributi completi sono nella sezione 4.8. PK indica la chiave primaria, FK un riferimento, U l'unicità di un attributo e ? la nullabilità. Le molteplicità descrivono la struttura; trigger e routine aggiungono le regole tra righe.

4.1 UML ristrutturato: account e profili

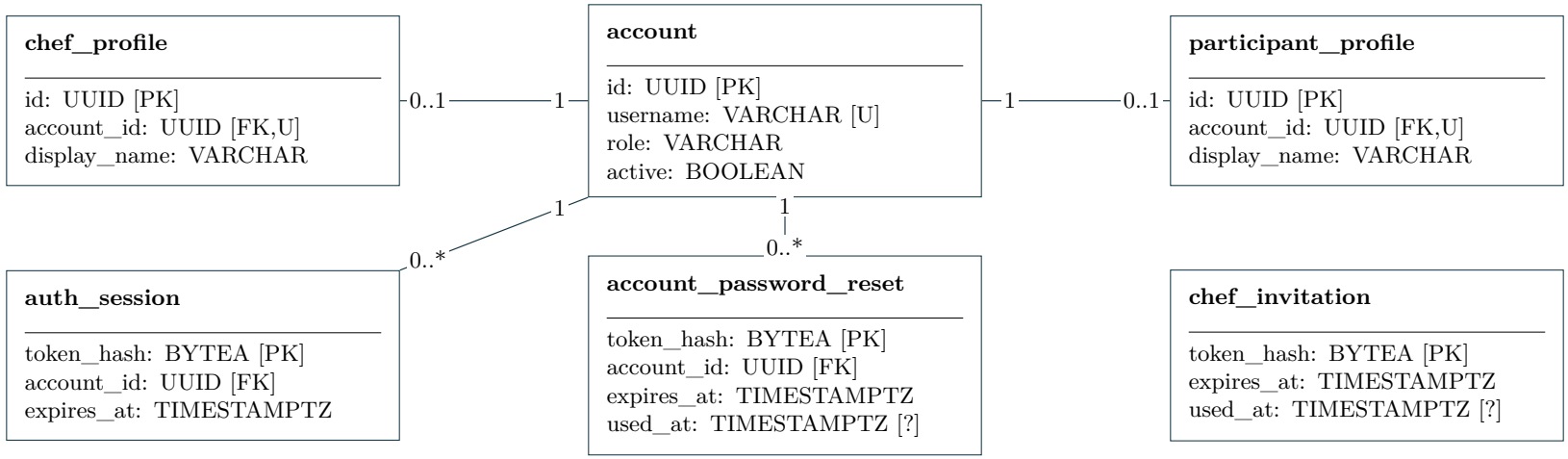


Figura 7: UML ristrutturato: account e profili

account_id UNIQUE limita ogni tipo a un profilo per account. auth_session supporta gli accessi correnti; account_password_reset e chef_invitation conservano dati storici. chef_invitation non ha FK. Routine e trigger controllano ruolo e creazione dei profili.

4.2 UML ristrutturato: corsi, iscrizioni e adesioni

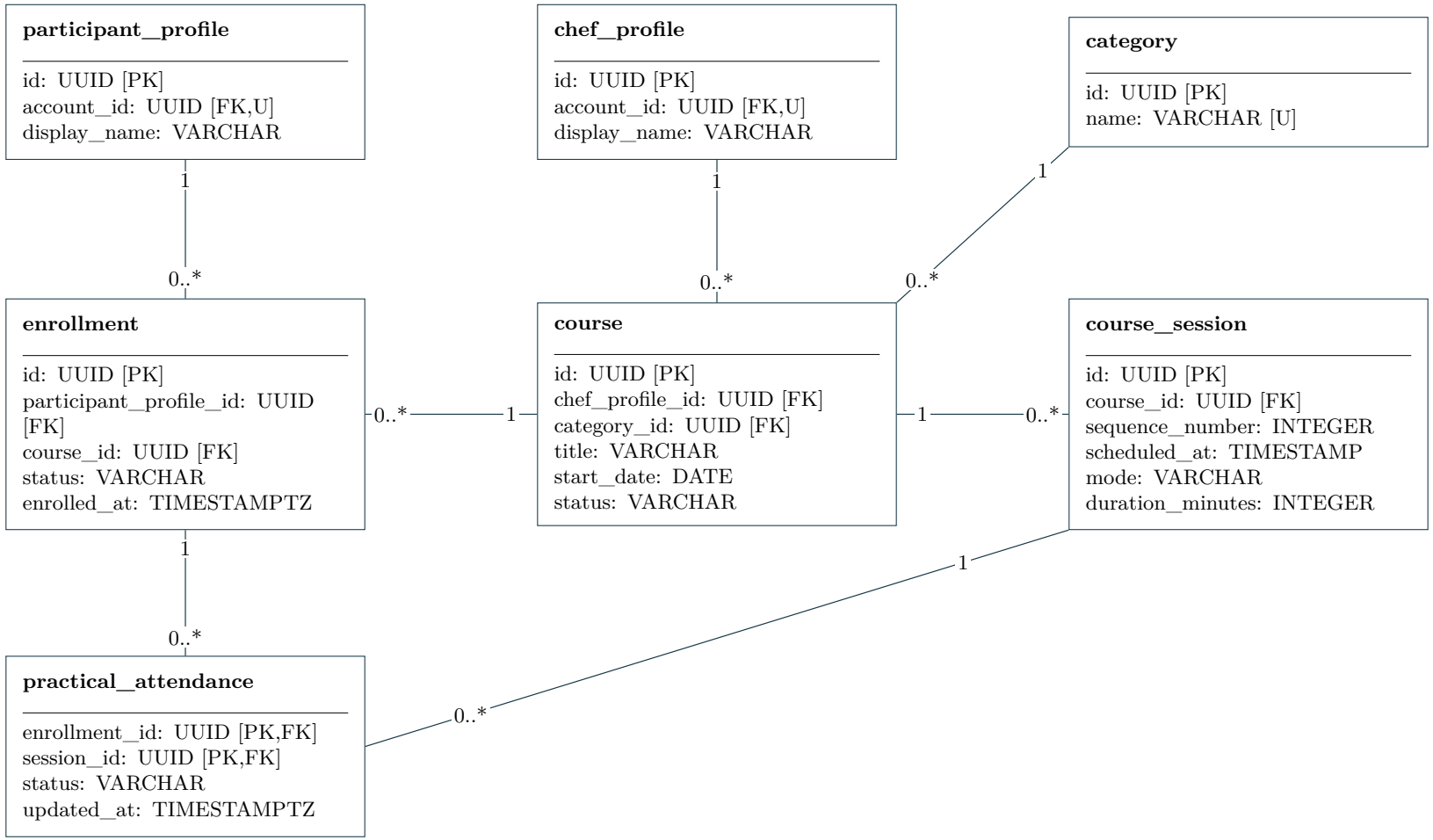


Figura 8: UML ristrutturato: corsi, iscrizioni e adesioni

La coppia partecipante/corso di enrollment è UNIQUE. Anche corso/numero della sessione è UNIQUE. La PK di practical_attendance è la coppia enrollment_id/session_id; il trigger verifica corso, iscrizione attiva e capienza. Le FK da sole consentono zero sessioni, mentre la creazione applicativa ne richiede inizialmente 2–30.

4.3 UML ristrutturato: ricettario, menu e copie

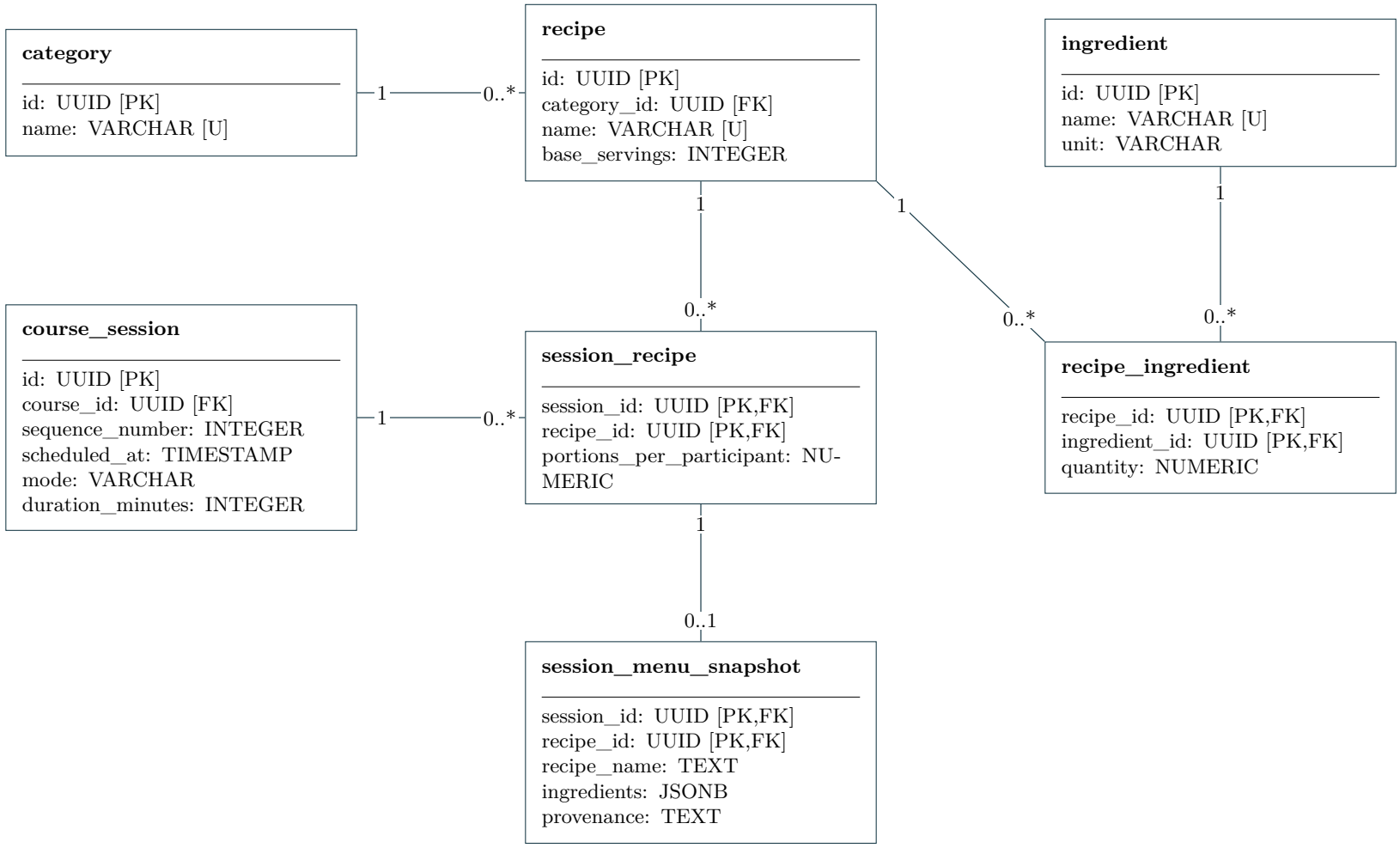


Figura 9: UML ristrutturato: ricettario, menu e copie

Le PK di recipe_ingredient e session_recipe sono coppie di UUID. La FK della copia usa insieme session_id e recipe_id. Essendo anche PK, quella coppia ammette al massimo una copia per associazione; il trigger la crea nel flusso ordinario. Le quantità NUMERIC mantengono la precisione delle dosi.

4.4 UML ristrutturato: comunicazioni e destinatari

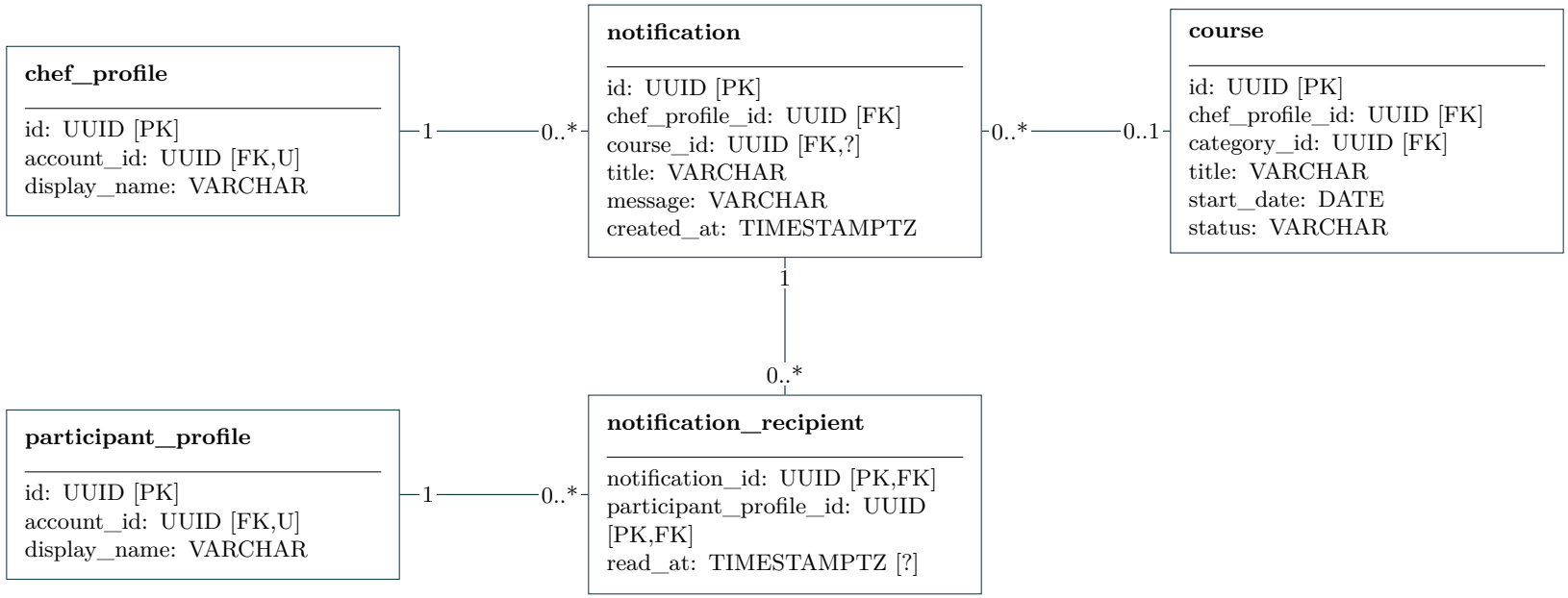


Figura 10: UML ristrutturato: comunicazioni e destinatari

notification.course_id è nullable: ogni messaggio può riguardare zero o un corso. La PK di notification_recipient è la coppia notification_id/participant_profile_id. read_at è opzionale, così un destinatario non ancora letto rimane comunque registrato.

4.5 UML ristrutturato: storico e supporto dello schema

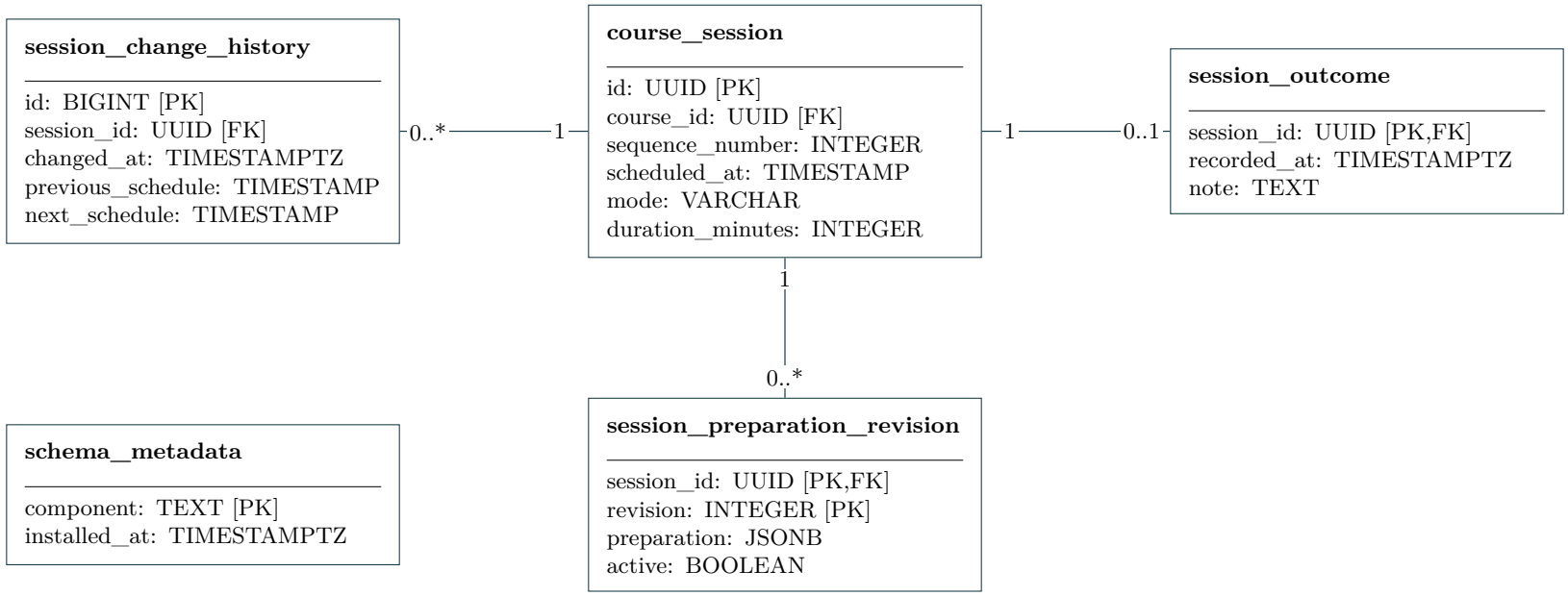


Figura 11: UML ristrutturato: storico e supporto dello schema

session_change_history registra ancora spostamenti e annullamenti tramite trigger. session_outcome e session_preparation_revision conservano esiti e revisioni storici, senza governare lo svolgimento o i fabbisogni correnti. La revisione è identificata da sessione/numero; un indice parziale limita a una la revisione attiva per sessione. schema_metadata è tecnica e non ha FK. Gli array UUID e i dati JSONB non aggiungono FK implicite.

4.6 Dizionario delle classi concettuali

Tabella 2: Dizionario delle classi concettuali

Classe	Significato	Tabella
Account	Identità autenticabile con ruolo, credenziale e attivazione.	`app.account`
Chef	Organizza corsi, menu, comunicazioni e report.	`app.chef_profile`
Partecipante	Si iscrive ai corsi e risponde alle pratiche.	`app.participant_profile`
Categoria	Classifica corsi e ricette.	`app.category`
Corso	Percorso tematico con calendario e frequenza.	`app.course`
Sessione	Appuntamento online o pratico con data, durata e dati specifici.	`app.course_session`
Ricetta	Ricetta corrente per una persona.	`app.recipe`
Ingrediente	Ingrediente con unità canonica.	`app.ingredient`
DoseRicetta	Quantità di un ingrediente in una ricetta.	`app.recipe_ingredient`
RicettaSessione	Ricetta associata al menu di una pratica.	`app.session_recipe`
CopiaMenu	Nome, descrizione e dosi acquisite per la sessione.	`app.session_menu_snapshot`
Iscrizione	Legame partecipante/corso con stato e periodo corrente.	`app.enrollment`
Adesione	Risposta esplicita alla pratica di un corso a cui si è iscritti.	`app.practical_attendance`
Comunicazione	Messaggio interno dello Chef per uno o tutti i corsi.	`app.notification`
Destinatario	Destinatario registrato con lettura opzionale.	`app.notification_recipient`

4.7 Dizionario delle associazioni

Le cardinalità esprimono quante istanze possono essere collegate; l'ultima colonna indica come il collegamento viene conservato e perché. Nella tabella 3 sono comprese anche le relazioni non mostrate nelle singole tavole.

Tabella 3: Dizionario delle associazioni

Associazione	Cardinalità	Realizzazione e motivazione
Account / Chef	1 : 0..1	account_id UNIQUE FK; trigger ruolo; profilo creato insieme ad account da routine
Account / Partecipante	1 : 0..1	account_id UNIQUE FK; trigger ruolo; profili incompatibili impediti dal ruolo
Chef / Corso	1 : 0..N	chef_profile_id FK
Categoria / Corso	1 : 0..N	category_id FK
Categoria / Ricetta	1 : 0..N	category_id FK

Continua nella pagina successiva

Associazione	Cardinalità	Realizzazione e motivazione
Corso / Sessione	1 : 0..N	course_id FK; creazione applicativa iniziale di 2–30 sessioni; il CHECK sul conteggio ammette 1–30, ma non impone il minimo di righe
Ricetta / DoseRicetta	1 : 0..N	PK ricetta/ingrediente
Ingrediente / DoseRicetta	1 : 0..N	ingrediente in più ricette
Sessione / RicettaSessione	1 : 0..N	solo pratica, più ricette consentite
Ricetta / RicettaSessione	1 : 0..N	recipe_id FK
RicettaSessione / CopiaMenu	1 : 0..1	FK composita; trigger inserimento produce copia ordinariamente
Partecipante / Iscrizione	1 : 0..N	FK, UNIQUE partecipante/corso
Corso / Iscrizione	1 : 0..N	FK
Iscrizione / Adesione	1 : 0..N	FK
Sessione / Adesione	1 : 0..N	FK, solo pratica stesso corso
Chef / Comunicazione	1 : 0..N	autore FK
Corso / Comunicazione	0..1 : 0..N	course_id nullable per messaggi globali
Comunicazione / Destinatario	1 : 0..N	FK
Partecipante / Destinatario	1 : 0..N	FK

4.8 Schema logico e dizionario degli attributi

Sono utilizzati identificatori UUID per gli oggetti referenziati da più relazioni, così una correzione del nome non cambia i collegamenti. I vincoli UNIQUE proteggono separatamente le identità di dominio, come lo username e la coppia partecipante/corso. Le associative senza identità autonoma usano la coppia delle FK come chiave primaria. NUMERIC e BigDecimal conservano le dosi esatte: un CHECK rifiuta oltre tre decimali invece di arrotondarli. Gli appuntamenti sono date e orari locali interpretati in Europe/Rome; accessi e comunicazioni usano timestamp con fuso per rappresentare istanti.

Nella tabella 4 ogni relazione elenca i propri attributi; PK indica la chiave primaria. Le FK e le chiavi alternative sono dettagliate subito dopo, insieme ai tipi e ai vincoli.

Tabella 4: Schema logico e dizionario degli attributi

Relazione e attributi	Chiave primaria
`app.account(id, username, password_hash, role, active, created_at)`	`PRIMARY KEY (id)`
`app.account_password_reset(token_hash, account_id, created_at, expires_at, used_at)`	`PRIMARY KEY (token_hash)`
`app.auth_session(token_hash, account_id, created_at, expires_at)`	`PRIMARY KEY (token_hash)`

Continua nella pagina successiva

Relazione e attributi	Chiave primaria
app.category(id, name, description)	PRIMARY KEY (id)
app.chef_invitation(token_hash, created_at, expires_at, used_at)	PRIMARY KEY (token_hash)
app.chef_profile(id, account_id, display_name)	PRIMARY KEY (id)
app.course(id, chef_profile_id, category_id, title, description, start_date, start_time, recurrence_days, session_count, status, created_at, ingredient_margin)	PRIMARY KEY (id)
app.course_session(id, course_id, sequence_number, scheduled_at, mode, status, duration_minutes, online_url, venue, capacity, attendance_closes_at, completed_at)	PRIMARY KEY (id)
app.enrollment(id, participant_profile_id, course_id, status, enrolled_at, withdrawn_at)	PRIMARY KEY (id)
app.ingredient(id, name, unit)	PRIMARY KEY (id)
app.notification(id, chef_profile_id, course_id, title, message, reason, created_at)	PRIMARY KEY (id)
app.notification_recipient(notification_id, participant_profile_id, read_at)	PRIMARY KEY (notification_id, participant_profile_id)
app.participant_profile(id, account_id, display_name)	PRIMARY KEY (id)
app.practical_attendance(enrollment_id, session_id, status, updated_at)	PRIMARY KEY (enrollment_id, session_id)
app.recipe(id, category_id, name, description, base_servings)	PRIMARY KEY (id)
app.recipe_ingredient(recipe_id, ingredient_id, quantity)	PRIMARY KEY (recipe_id, ingredient_id)
app.schema_metadata(component, installed_at)	PRIMARY KEY (component)
app.session_change_history(id, session_id, changed_at, previous_schedule, next_schedule, previous_status, next_status, previous_attendance)	PRIMARY KEY (id)
app.session_menu_snapshot(session_id, recipe_id, recipe_name, description, ingredients, captured_at, provenance)	PRIMARY KEY (session_id, recipe_id)
app.session_outcome(session_id, recorded_at, recipe_ids, enrollment_ids, note)	PRIMARY KEY (session_id)
app.session_preparation_revision(session_id, revision, created_at, reason, preparation, confirmed_enrollments, active)	PRIMARY KEY (session_id, revision)
app.session_recipe(session_id, recipe_id, portions_per_participant)	PRIMARY KEY (session_id, recipe_id)

La struttura delle tabelle è definita dalla migrazione iniziale e rimane invariata nella migrazione correttiva. Comprende 22 tabelle applicative e 115 colonne; la tabella di cronologia Flyway è metadato del framework e non viene contata. Per ciascuna relazione sono indicati attributi, tipi, nullabilità e chiavi. Nella colonna NULL, sì indica un attributo facoltativo e no indica NOT NULL. Il trattino nella colonna Default indica l'assenza di un valore predefinito.

4.9 app.account

Classificazione: operativa. Identità autenticabile con ruolo, credenziale e attivazione.

Tabella 5: Attributi di app.account

Attributo	Tipo fisico	NULL	Default
id	uuid	no	gen_random_uuid()
username	character varying(50)	no	-
password_hash	text	no	-
role	character varying(20)	no	-
active	boolean	no	true
created_at	timestamp with time zone	no	clock_timestamp()

PRIMARY KEY (id)

UNIQUE (username)

4.10 app.account_password_reset

Classificazione: storica. Archivio dei token di recupero, senza operazioni disponibili nel client.

Tabella 6: Attributi di app.account_password_reset

Attributo	Tipo fisico	NULL	Default
token_hash	bytea	no	-
account_id	uuid	no	-
created_at	timestamp with time zone	no	clock_timestamp()
expires_at	timestamp with time zone	no	-
used_at	timestamp with time zone	sì	-

FOREIGN KEY (account_id) REFERENCES app.account(id)

PRIMARY KEY (token_hash)

4.11 app.auth_session

Classificazione: tecnica. Sessioni autenticate: hash del token, account e intervallo di validità.

Tabella 7: Attributi di app.auth_session

Attributo	Tipo fisico	NULL	Default
token_hash	bytea	no	-
account_id	uuid	no	-
created_at	timestamp with time zone	no	clock_timestamp()
expires_at	timestamp with time zone	no	-

```
FOREIGN KEY (account_id) REFERENCES app.account(id) ON DELETE CASCADE
PRIMARY KEY (token_hash)
```

4.12 app.category

Classificazione: operativa. Classifica corsi e ricette.

Tabella 8: Attributi di app.category

Attributo	Tipo fisico	NULL	Default
id	uuid	no	gen_random_uuid()
name	character varying(80)	no	-
description	text	no	-

```
UNIQUE (name)
PRIMARY KEY (id)
```

4.13 app.chef_invitation

Classificazione: storica. Archivio dei codici di invito, senza operazioni disponibili nel client.

Tabella 9: Attributi di app.chef_invitation

Attributo	Tipo fisico	NULL	Default
token_hash	bytea	no	-
created_at	timestamp with time zone	no	clock_timestamp()
expires_at	timestamp with time zone	no	-
used_at	timestamp with time zone	sì	-

```
PRIMARY KEY (token_hash)
```

4.14 app.chef_profile

Classificazione: operativa. Organizza corsi, menu, comunicazioni e report.

Tabella 10: Attributi di app.chef_profile

Attributo	Tipo fisico	NULL	Default
id	uuid	no	gen_random_uuid()
account_id	uuid	no	–
display_name	character varying(100)	no	–

```
FOREIGN KEY (account_id) REFERENCES app.account(id)
UNIQUE (account_id)
PRIMARY KEY (id)
```

4.15 app.course

Classificazione: operativa. Percorso tematico con calendario e frequenza.

Tabella 11: Attributi di app.course

Attributo	Tipo fisico	NULL	Default
id	uuid	no	gen_random_uuid()
chef_profile_id	uuid	no	–
category_id	uuid	no	–
title	character varying(120)	no	–
description	text	no	–
start_date	date	no	–
start_time	time without time zone	no	–
recurrence_days	integer	no	–
session_count	integer	no	–
status	character varying(20)	no	'DRAFT'::character varying
created_at	timestamp with time zone	no	clock_timestamp()
ingredient_margin	numeric	no	0

```
FOREIGN KEY (category_id) REFERENCES app.category(id)
FOREIGN KEY (chef_profile_id) REFERENCES app.chef_profile(id)
UNIQUE (chef_profile_id, title, start_date)
PRIMARY KEY (id)
```

4.16 app.course_session

Classificazione: operativa. Appuntamento online o pratico con data, durata e dati specifici.

Tabella 12: Attributi di app.course_session

Attributo	Tipo fisico	NULL	Default
id	uuid	no	gen_random_uuid()
course_id	uuid	no	–
sequence_number	integer	no	–
scheduled_at	timestamp without time zone	no	–
mode	character varying(20)	no	–
status	character varying(20)	no	'SCHEDULED'::character varying
duration_minutes	integer	no	120
online_url	text	sì	–
venue	character varying(180)	sì	–
capacity	integer	sì	–
attendance_closes_at	timestamp without time zone	sì	–
completed_at	timestamp with time zone	sì	–

```
FOREIGN KEY (course_id) REFERENCES app.course(id) ON DELETE CASCADE
PRIMARY KEY (id)
UNIQUE (course_id, sequence_number)
```

4.17 app.enrollment

Classificazione: operativa. Legame partecipante/corso con stato e periodo corrente.

Tabella 13: Attributi di app.enrollment

Attributo	Tipo fisico	NULL	Default
id	uuid	no	gen_random_uuid()
participant_profile_id	uuid	no	–
course_id	uuid	no	–
status	character varying(20)	no	–
enrolled_at	timestamp with time zone	no	clock_timestamp()
withdrawn_at	timestamp with time zone	sì	–

```
FOREIGN KEY (course_id) REFERENCES app.course(id)
```

UNIQUE (participant_profile_id, course_id)

FOREIGN KEY (participant_profile_id) REFERENCES app.participant_profile(id)

PRIMARY KEY (id)

4.18 app.ingredient

Classificazione: operativa. Ingrediente con unità canonica.

Tabella 14: Attributi di app.ingredient

Attributo	Tipo fisico	NULL	Default
id	uuid	no	gen_random_uuid()
name	character varying(100)	no	-
unit	character varying(20)	no	-

UNIQUE (name)

PRIMARY KEY (id)

4.19 app.notification

Classificazione: operativa. Messaggio interno dello Chef per uno o tutti i corsi.

Tabella 15: Attributi di app.notification

Attributo	Tipo fisico	NULL	Default
id	uuid	no	gen_random_uuid()
chef_profile_id	uuid	no	-
course_id	uuid	sì	-
title	character varying(120)	no	-
message	character varying(2000)	no	-
reason	character varying(30)	no	-
created_at	timestamp with time zone	no	clock_timestamp()

FOREIGN KEY (chef_profile_id) REFERENCES app.chef_profile(id)

FOREIGN KEY (course_id) REFERENCES app.course(id)

PRIMARY KEY (id)

4.20 app.notification_recipient

Classificazione: operativa. Destinatario registrato con lettura opzionale.

Tabella 16: Attributi di app.notification_recipient

Attributo	Tipo fisico	NULL	Default
notification_id	uuid	no	-
participant_profile_id	uuid	no	-
read_at	timestamp with time zone	sì	-

```
FOREIGN KEY (notification_id) REFERENCES app.notification(id) ON DELETE CASCADE
FOREIGN KEY (participant_profile_id) REFERENCES app.participant_profile(id)
PRIMARY KEY (notification_id, participant_profile_id)
```

4.21 app.participant_profile

Classificazione: operativa. Si iscrive ai corsi e risponde alle pratiche.

Tabella 17: Attributi di app.participant_profile

Attributo	Tipo fisico	NULL	Default
id	uuid	no	gen_random_uuid()
account_id	uuid	no	-
display_name	character varying(100)	no	-

```
FOREIGN KEY (account_id) REFERENCES app.account(id)
UNIQUE (account_id)
PRIMARY KEY (id)
```

4.22 app.practical_attendance

Classificazione: operativa. Risposta esplicita alla pratica di un corso a cui si è iscritti.

Tabella 18: Attributi di app.practical_attendance

Attributo	Tipo fisico	NULL	Default
enrollment_id	uuid	no	-
session_id	uuid	no	-
status	character varying(20)	no	-
updated_at	timestamp with time zone	no	clock_timestamp()

```
FOREIGN KEY (enrollment_id) REFERENCES app.enrollment(id)
```

```
PRIMARY KEY (enrollment_id, session_id)
FOREIGN KEY (session_id) REFERENCES app.course_session(id)
```

4.23 app.recipe

Classificazione: operativa. Ricetta corrente per una persona.

Tabella 19: Attributi di app.recipe

Attributo	Tipo fisico	NULL	Default
id	uuid	no	gen_random_uuid()
category_id	uuid	no	-
name	character varying(120)	no	-
description	text	no	-
base_servings	integer	no	-

```
FOREIGN KEY (category_id) REFERENCES app.category(id)
UNIQUE (name)
PRIMARY KEY (id)
```

4.24 app.recipe_ingredient

Classificazione: operativa. Quantità di un ingrediente in una ricetta.

Tabella 20: Attributi di app.recipe_ingredient

Attributo	Tipo fisico	NULL	Default
recipe_id	uuid	no	-
ingredient_id	uuid	no	-
quantity	numeric	no	-

```
FOREIGN KEY (ingredient_id) REFERENCES app.ingredient(id)
PRIMARY KEY (recipe_id, ingredient_id)
FOREIGN KEY (recipe_id) REFERENCES app.recipe(id) ON DELETE CASCADE
```

4.25 app.schema_metadata

Classificazione: tecnica. Marcatore tecnico della fondazione dello schema.

Tabella 21: Attributi di app.schema_metadata

Attributo	Tipo fisico	NULL	Default
component	text	no	-

Continua nella pagina successiva

Attributo	Tipo fisico	NULL	Default
installed_at	timestamp with time zone	no	clock_timestamp()

PRIMARY KEY (component)

4.26 app.session_change_history

Classificazione: storica. Cronologia del calendario ancora alimentata: il trigger preserve_session_change richiama la funzione omonima e registra nuovi spostamenti o annullamenti. È distinta dagli archivi di invito, recupero, revisioni ed esiti non più gestiti dai flussi correnti.

Tabella 22: Attributi di app.session_change_history

Attributo	Tipo fisico	NULL	Default
id	bigint	no	-
session_id	uuid	no	-
changed_at	timestamp with time zone	no	clock_timestamp()
previous_schedule	timestamp without time zone	no	-
next_schedule	timestamp without time zone	no	-
previous_status	text	no	-
next_status	text	no	-
previous_attendance	jsonb	no	-

PRIMARY KEY (id)

FOREIGN KEY (session_id) REFERENCES app.course_session(id)

4.27 app.session_menu_snapshot

Classificazione: operativa. Nome, descrizione e dosi acquisite per la sessione.

Tabella 23: Attributi di app.session_menu_snapshot

Attributo	Tipo fisico	NULL	Default
session_id	uuid	no	-
recipe_id	uuid	no	-
recipe_name	text	no	-
description	text	no	-
ingredients	jsonb	no	-

Continua nella pagina successiva

Attributo	Tipo fisico	NULL	Default
captured_at	timestamp with time zone	no	clock_timestamp()
provenance	text	no	-

PRIMARY KEY (session_id, recipe_id)

FOREIGN KEY (session_id, recipe_id) REFERENCES app.session_recipe(session_id, recipe_id) ON DELETE CASCADE

4.28 app.session_outcome

Classificazione: storica. Esiti manuali conservati come storico; lo stato corrente è derivato dal calendario.

Tabella 24: Attributi di app.session_outcome

Attributo	Tipo fisico	NULL	Default
session_id	uuid	no	-
recorded_at	timestamp with time zone	no	clock_timestamp()
recipe_ids	uuid[]	no	-
enrollment_ids	uuid[]	sì	-
note	text	no	-

PRIMARY KEY (session_id)

FOREIGN KEY (session_id) REFERENCES app.course_session(id)

4.29 app.session_preparation_revision

Classificazione: storica. Revisioni della preparazione conservate come storico; il fabbisogno corrente usa menu e adesioni valide.

Tabella 25: Attributi di app.session_preparation_revision

Attributo	Tipo fisico	NULL	Default
session_id	uuid	no	-
revision	integer	no	-
created_at	timestamp with time zone	no	clock_timestamp()
reason	text	no	-
preparation	jsonb	no	-
confirmed_enrollments	uuid[]	no	-
active	boolean	no	true

PRIMARY KEY (session_id, revision)

FOREIGN KEY (session_id) REFERENCES app.course_session(id)

4.30 app.session_recipe

Classificazione: operativa. Ricetta associata al menu di una pratica.

Tabella 26: Attributi di app.session_recipe

Attributo	Tipo fisico	NULL	Default
session_id	uuid	no	-
recipe_id	uuid	no	-
portions_per_participant	numeric	no	1

```
PRIMARY KEY (session_id, recipe_id)
```

```
FOREIGN KEY (recipe_id) REFERENCES app.recipe(id)
```

```
FOREIGN KEY (session_id) REFERENCES app.course_session(id) ON DELETE CASCADE
```

4.31 Dizionario dei vincoli

Il catalogo contiene 205 vincoli: 22 chiavi primarie, 24 chiavi esterne, 9 unicità, 46 CHECK e 104 NOT NULL. Chiavi e nullabilità sono descritte insieme alle relazioni nella sezione 4.8. Per evitare di ripetere il codice SQL, il dizionario seguente esprime in parole le condizioni dei CHECK ed è raggruppato per tabella. I nomi e le espressioni complete sono in `src/main/resources/db/migration/V001__create_initial_schema.sql`. I controlli tra più righe sono approfonditi nella sezione 7.2.

4.31.1 account

Tabella 27: Vincoli di account

Vincolo	Condizione e significato
account_password_hash_format	L'hash inizia con il prefisso $pbkdf2-sha256$600000$; il CHECK verifica il prefisso del formato, senza validare il contenuto successivo.
account_role_valid	Il ruolo ammette soltanto CHEF o PARTICIPANT.
account_username_format	Lo username è minuscolo, lungo da 3 a 50 caratteri e contiene soltanto lettere da a a z, cifre, punto, trattino basso o trattino.

4.31.2 account_password_reset

Tabella 28: Vincoli di account_password_reset

Vincolo	Condizione e significato
account_password_reset_expiry_valid	La scadenza del token è successiva alla sua creazione.

Continua nella pagina successiva

Vincolo	Condizione e significato
account_password_reset_used_after_creation	La data di utilizzo può essere assente; quando presente, non precede la creazione del token.

4.31.3 auth_session

Tabella 29: Vincoli di auth_session

Vincolo	Condizione e significato
auth_session_expiry_valid	La scadenza della sessione di autenticazione è successiva alla sua creazione.

4.31.4 category

Tabella 30: Vincoli di category

Vincolo	Condizione e significato
category_description_length_check	La descrizione, esclusi gli spazi iniziali e finali, contiene al massimo 500 caratteri; è ammessa una descrizione vuota.
category_name_check	Il nome non è vuoto dopo la rimozione degli spazi iniziali e finali.
category_name_length_check	Il nome, esclusi gli spazi iniziali e finali, contiene da 3 a 80 caratteri.

4.31.5 chef_invitation

Tabella 31: Vincoli di chef_invitation

Vincolo	Condizione e significato
chef_invitation_expiry_valid	La scadenza dell'invito è successiva alla sua creazione.
chef_invitation_used_after_creation	La data di utilizzo può essere assente; quando presente, non precede la creazione dell'invito.

4.31.6 chef_profile

Tabella 32: Vincoli di chef_profile

Vincolo	Condizione e significato
chef_profile_display_name_check	Il nome visualizzato non è vuoto dopo la rimozione degli spazi iniziali e finali.

4.31.7 course

Tabella 33: Vincoli di course

Vincolo	Condizione e significato
course_description_valid	La descrizione non è vuota dopo la rimozione degli spazi iniziali e finali.
course_ingredient_margin_valid	Il margine degli ingredienti è compreso tra 0 e 25, estremi inclusi.
course_no_ingredient_margin	Il margine degli ingredienti è uguale a 0; questo vincolo restringe ulteriormente l'intervallo ammesso dall'altro CHECK sul margine.
course_recurrence_valid	L'intervallo tra le sessioni è compreso tra 1 e 31 giorni, estremi inclusi.
course_session_count_valid	Il numero delle sessioni è compreso tra 1 e 30, estremi inclusi.
course_status_valid	Lo stato ammette soltanto DRAFT, PUBLISHED, COMPLETED o CANCELLED.
course_title_valid	Il titolo contiene da 3 a 30 caratteri, senza spazi iniziali o finali. Sono esclusi ritorno carrello, nuova riga, tabulazione verticale, avanzamento pagina e separatori Unicode U+0085, U+2028 e U+2029.

4.31.8 course_session

Tabella 34: Vincoli di course_session

Vincolo	Condizione e significato
course_session_attendance_cutoff_valid	Per ONLINE la scadenza delle adesioni è assente; per PRACTICAL è presente e precede strettamente l'inizio programmato.
course_session_duration_valid	La durata è compresa tra 1 e 720 minuti, estremi inclusi.
course_session_mode_check	La modalità ammette soltanto ONLINE o PRACTICAL.
course_session_mode_fields_valid	Per ONLINE il link è presente, mentre sede e capienza sono assenti. Il link rispetta il formato HTTPS senza distinzione tra maiuscole e minuscole: host con primo carattere alfanumerico e successivi alfanumerici, punti o trattini; porta numerica facoltativa; eventuale parte finale introdotta da /, ? o # e senza caratteri di spaziatura. Per PRACTICAL il link è assente, la sede è presente, non vuota e senza spazi iniziali o finali, e la capienza è presente e positiva.
course_session_sequence_number_check	Il numero progressivo della sessione è maggiore di 0.
course_session_status_check	Lo stato memorizzato ammette soltanto SCHEDULED, COMPLETED o CANCELLED.

4.31.9 enrollment

Tabella 35: Vincoli di enrollment

Vincolo	Condizione e significato
`enrollment_status_valid`	Lo stato dell'iscrizione ammette soltanto ACTIVE o WITHDRAWN.
`enrollment_withdrawal_valid`	Per ACTIVE la data di ritiro è assente; per WITHDRAWN è presente.

4.31.10 ingredient

Tabella 36: Vincoli di ingredient

Vincolo	Condizione e significato
`ingredient_name_check`	Il nome non è vuoto dopo la rimozione degli spazi iniziali e finali.
`ingredient_unit_valid`	L'unità ammette soltanto GRAM, MILLILITER, PIECE o TEASPOON.

4.31.11 notification

Tabella 37: Vincoli di notification

Vincolo	Condizione e significato
`notification_message_valid`	Il messaggio contiene da 3 a 2000 caratteri, senza spazi iniziali o finali.
`notification_reason_valid`	Il motivo ammette soltanto SCHEDULE_CHANGED, SESSION_CANCELLED, SESSION_ADDED o GENERAL.
`notification_title_valid`	Il titolo contiene da 3 a 120 caratteri, senza spazi iniziali o finali.

4.31.12 participant_profile

Tabella 38: Vincoli di participant_profile

Vincolo	Condizione e significato
`participant_profile_display_name_check`	Il nome visualizzato non è vuoto dopo la rimozione degli spazi iniziali e finali.

4.31.13 practical_attendance

Tabella 39: Vincoli di practical_attendance

Vincolo	Condizione e significato
`practical_attendance_status_valid`	La risposta alla sessione pratica ammette soltanto CONFIRMED o DECLINED.

4.31.14 recipe

Tabella 40: Vincoli di recipe

Vincolo	Condizione e significato
`recipe_base_servings_check`	Il numero delle porzioni di base è maggiore di 0.
`recipe_description_check`	La descrizione non è vuota dopo la rimozione degli spazi iniziali e finali.
`recipe_name_check`	Il nome non è vuoto dopo la rimozione degli spazi iniziali e finali.
`recipe_single_portion`	Il numero delle porzioni di base è uguale a 1.

4.31.15 recipe_ingredient

Tabella 41: Vincoli di recipe_ingredient

Vincolo	Condizione e significato
`recipe_ingredient_quantity_check`	La quantità dell'ingrediente è maggiore di 0.
`recipe_quantity_precision`	La quantità è maggiore di 0 e minore di 1000000000000; il valore coincide con il suo arrotondamento a 3 decimali, quindi non sono ammesse cifre decimali significative oltre la terza.

4.31.16 session_menu_snapshot

Tabella 42: Vincoli di session_menu_snapshot

Vincolo	Condizione e significato
`session_menu_snapshot_provenance_check`	La provenienza della copia ammette soltanto CAPTURED o LEGACY_RECONSTRUCTED.

4.31.17 session_outcome

Tabella 43: Vincoli di session_outcome

Vincolo	Condizione e significato
`session_outcome_note_check`	La nota, esclusi gli spazi iniziali e finali, contiene al massimo 2000 caratteri; è ammessa una nota vuota.

4.31.18 session_preparation_revision

Tabella 44: Vincoli di session_preparation_revision

Vincolo	Condizione e significato
`session_preparation_ revision_reason_check`	La motivazione, esclusi gli spazi iniziali e finali, contiene da 1 a 2000 caratteri.
`session_preparation_ revision_revision_check`	Il numero della revisione è maggiore di 0.

4.31.19 session_recipe

Tabella 45: Vincoli di session_recipe

Vincolo	Condizione e significato
`session_recipe_portions_ valid`	Le porzioni per partecipante sono maggiori di 0.
`session_recipe_single_ portion`	Le porzioni per partecipante sono uguali a 1.

Le dosi positive con massimo tre decimali sono verificate senza arrotondamento implicito. Il limite del conteggio salvato nel corso non impone da solo il numero di righe delle sessioni: il caso d'uso di creazione richiede inizialmente 2–30 sessioni.

5 Componenti applicativi e classi di design

Il progetto usa Java 25, JavaFX 26.0.2, PostgreSQL 18.4, JDBC, HikariCP e Flyway. L'architettura comprende quattro livelli: presentation, application, domain e infrastructure. I controller raccolgono gli input e mostrano gli esiti; i casi d'uso coordinano le operazioni tramite interfacce; il dominio contiene valori e regole indipendenti da GUI e JDBC; l'infrastruttura implementa l'accesso ai dati.

ApplicationRuntime costruisce manualmente i componenti. Il package application.bootstrap dipende dall'infrastruttura perché ne collega le implementazioni: è un'eccezione esplicita, controllata dai test di architettura. Non vengono usati un ORM, un contenitore di dependency injection o un backend HTTP.

Il diagramma di design è organizzato in tavole complementari. Le figure da 12 a 15 dettagliano creazione del corso, valori del calendario, adesioni e pianificazione; le tavole successive collegano le classi principali delle altre aree operative. Sono classi, record e interfacce presenti nel codice. La suddivisione rende leggibili membri e relazioni; uno stesso nome ripetuto indica lo stesso tipo. Il concetto Corso non corrisponde a una classe Java persistente: è rappresentato da comandi, modelli di lettura e tabelle SQL.

La notazione dei membri usa + per pubblico, - per privato e tilde per visibilità di package. Nei record, i componenti sono valori immutabili con accessor; non sono campi pubblici mutabili. La freccia triangolare tratteggiata rappresenta la realizzazione di un'interfaccia; le altre frecce indicano riferimenti o dipendenze.

Le tavole presentano le classi principali delle aree operative. Sono omesse le classi di supporto grafico e i dettagli interni non necessari per comprendere i collegamenti. Le ellissi nei metodi omettono soltanto l'elenco dei parametri. Le frecce continue indicano riferimenti conservati negli oggetti; quelle tratteggiate con punta aperta indicano l'uso di un tipo; quelle con triangolo vuoto indicano la realizzazione di un'interfaccia. Le ripetizioni di uno stesso nome indicano la stessa classe nelle diverse tavole.

5.1 Creazione del corso e coordinamento

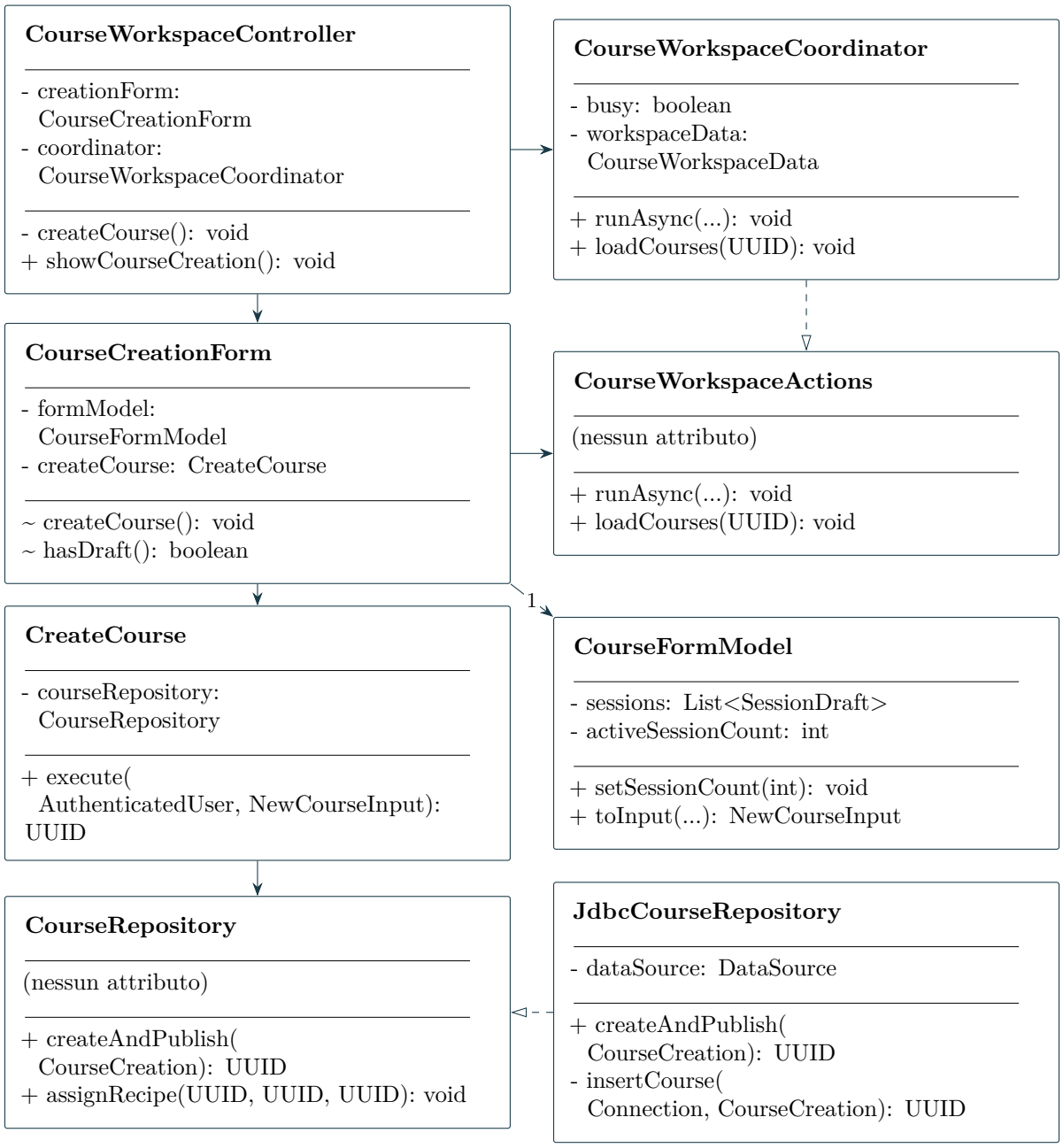


Figura 12: Creazione del corso e coordinamento

La tavola mostra i collegamenti principali del modulo; gli attributi e i metodi mostrati sono una selezione dei membri reali.

5.2 Valori del corso e calendario

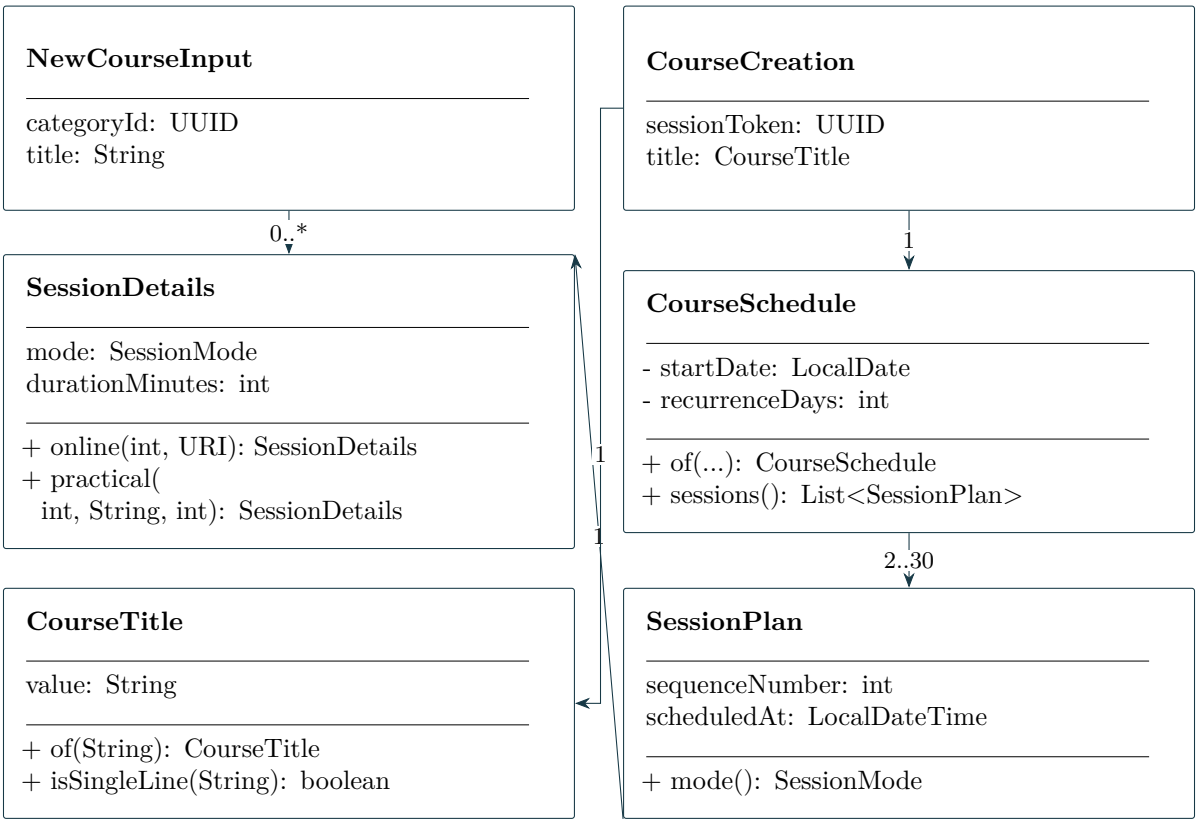


Figura 13: Valori del corso e calendario

La tavola mostra i collegamenti principali del modulo; gli attributi e i metodi mostrati sono una selezione dei membri reali.

Il caso d'uso CreateCourse della figura 12 riceve NewCourseInput, costruisce il comando CourseCreation della figura 13 e lo inoltra alla porta CourseRepository. Il comando contiene il titolo validato e il calendario, evitando di passare alla persistenza valori non controllati.

5.3 Gestione delle adesioni

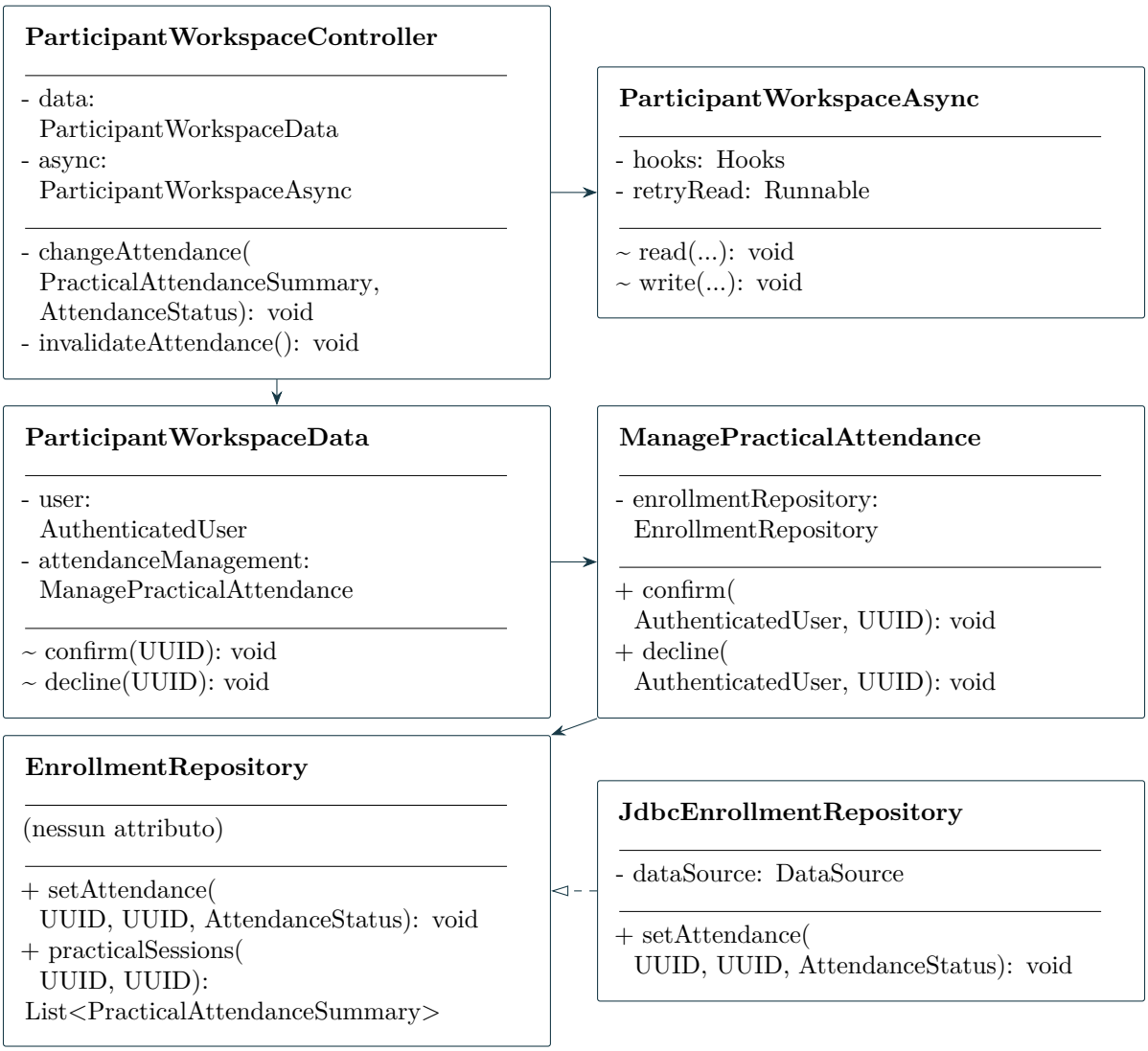


Figura 14: Gestione delle adesioni

La tavola mostra i collegamenti principali del modulo; gli attributi e i metodi mostrati sono una selezione dei membri reali.

5.4 Lettura della pianificazione

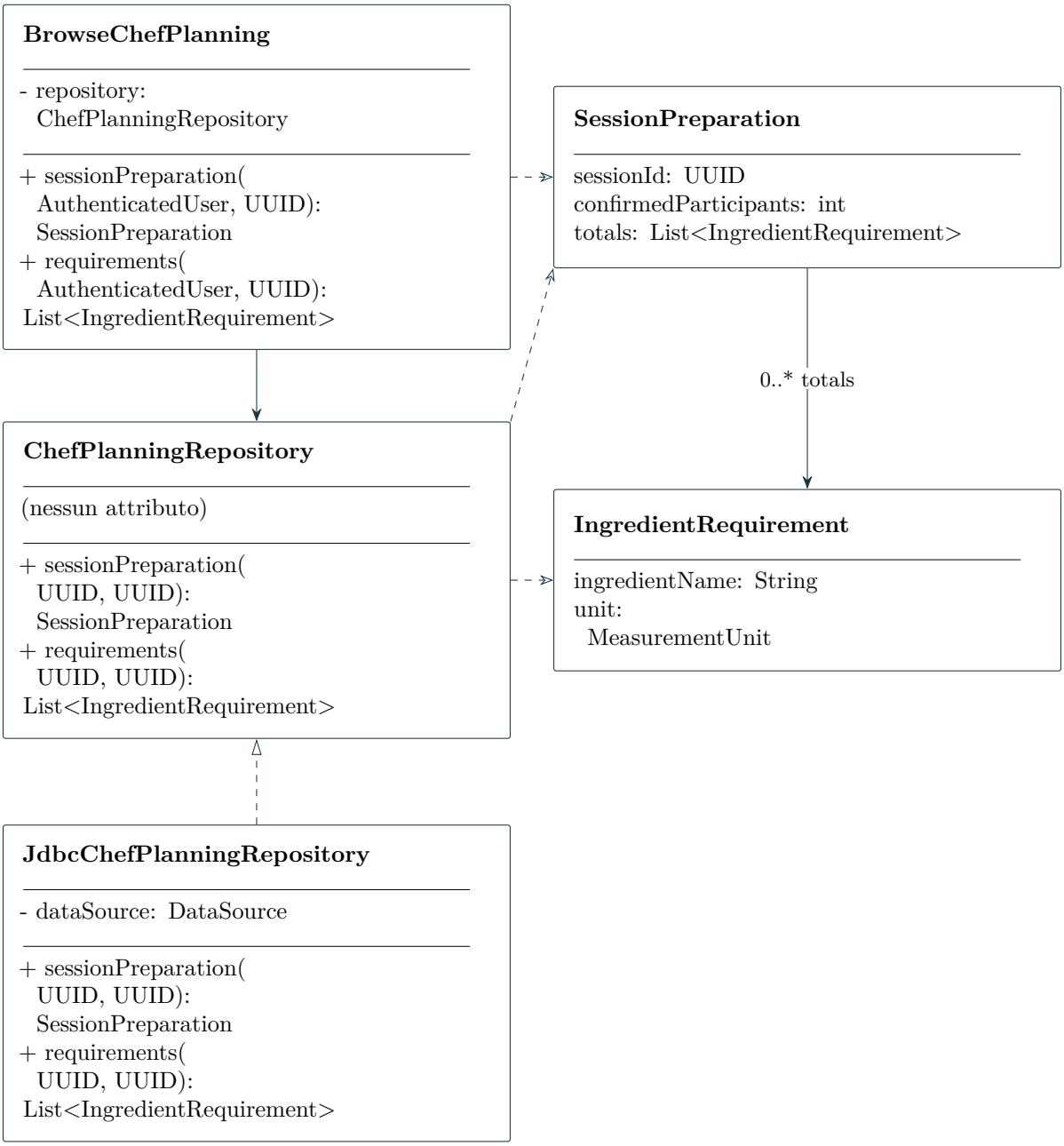


Figura 15: Lettura della pianificazione

La tavola mostra i collegamenti principali del modulo; gli attributi e i metodi mostrati sono una selezione dei membri reali.

Le transazioni sono nei repository JDBC. Il pool disattiva la conferma automatica e ammette 10 connessioni con attesa massima di 10 secondi. Creazione e pubblicazione sono atomiche. L'esportazione dei fabbisogni usa REPEATABLE_READ per mantenere letture coerenti.

5.5 Accesso, registrazione e gestione dell'account

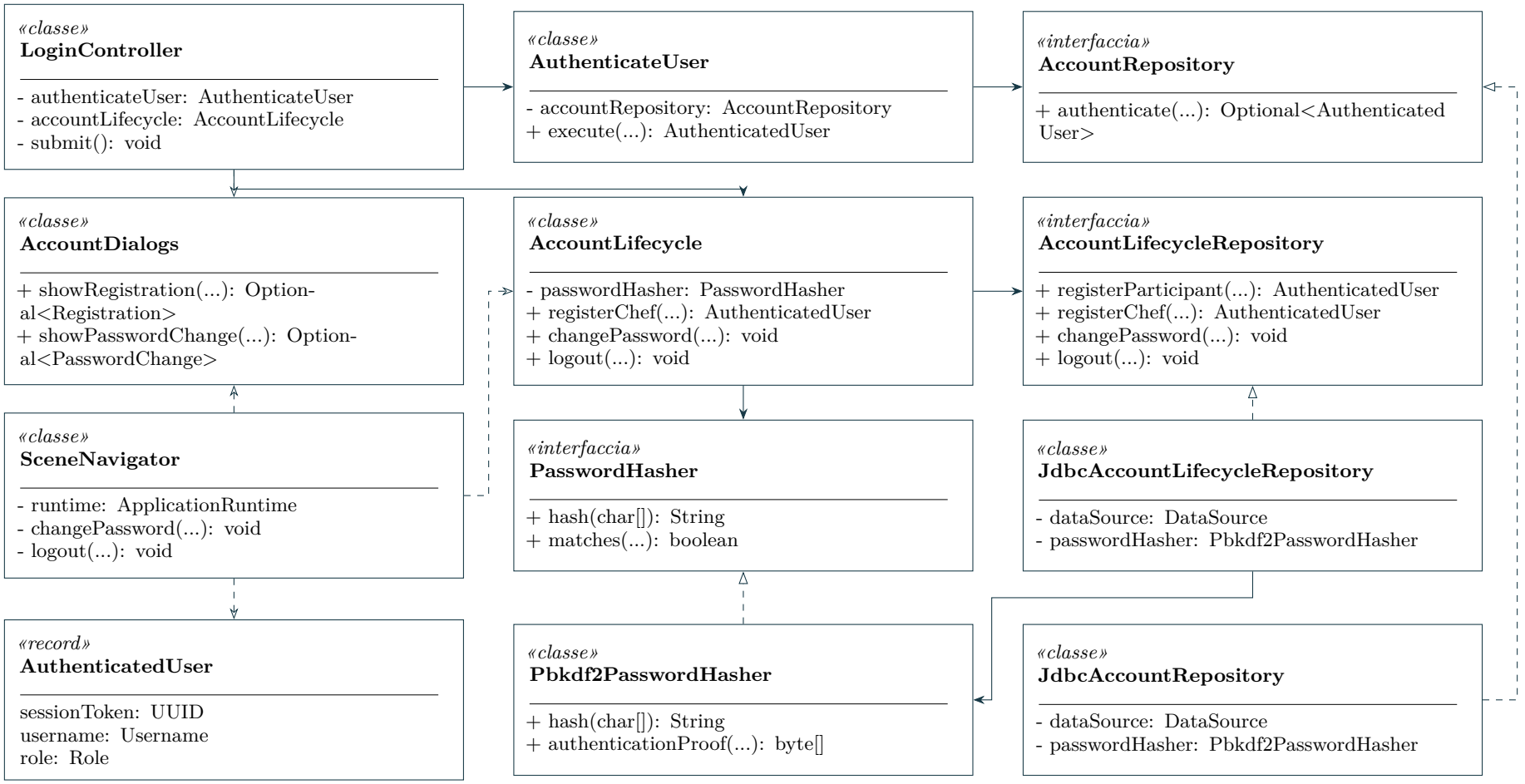


Figura 16: Accesso, registrazione e gestione dell'account

La sessione autenticata viene passata ai casi d'uso. SceneNavigator richiama AccountLifecycle attraverso ApplicationRuntime; i dialoghi raccolgono i dati, senza accedere direttamente al database.

5.6 Ricettario, ingredienti e categorie

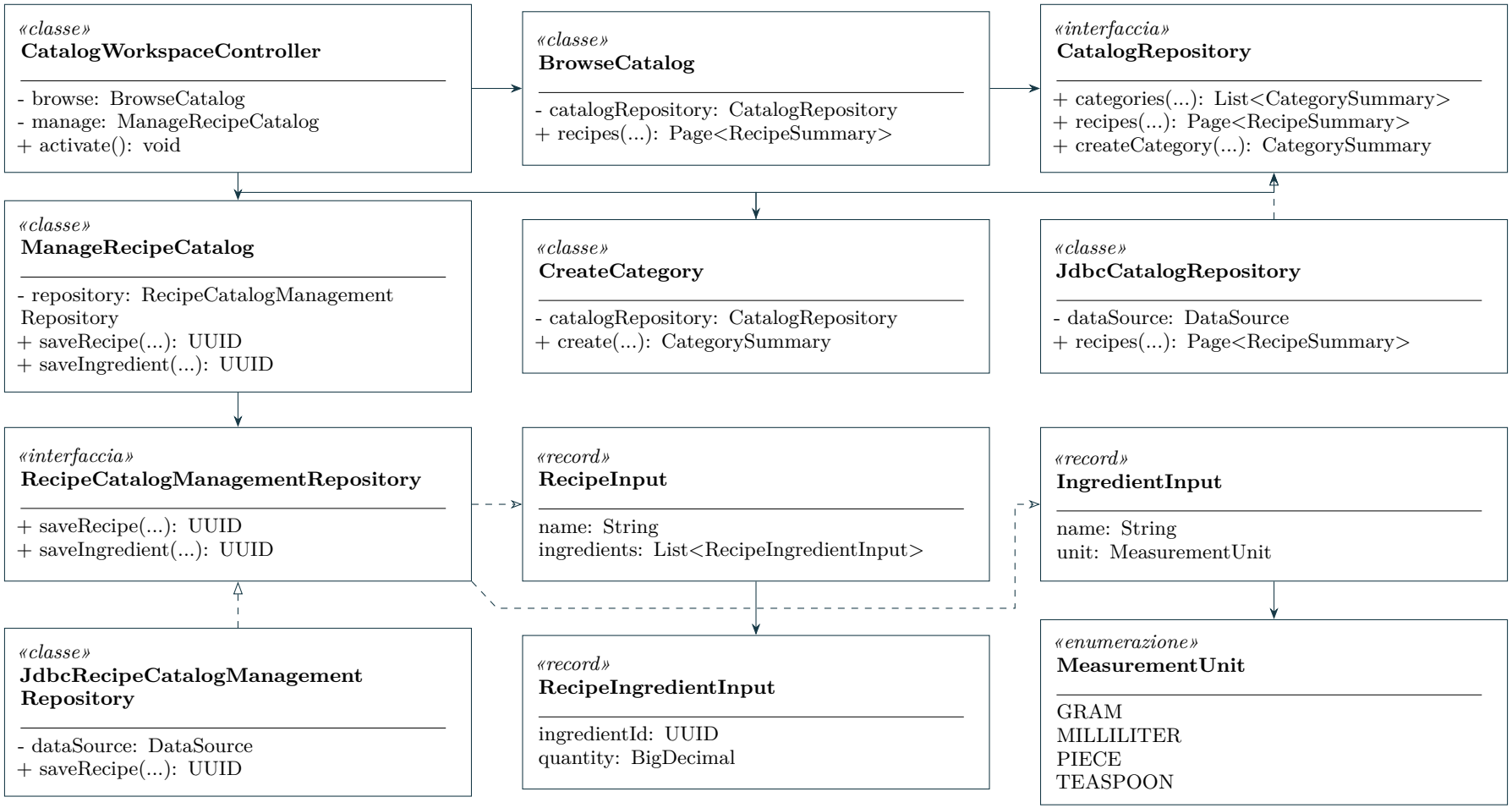


Figura 17: Ricettario, ingredienti e categorie

La lettura del catalogo e la sua modifica usano contratti separati. RecipeInput contiene le dosi per una persona; CreateCategory condivide CatalogRepository con BrowseCatalog.

5.7 Calendario dei corsi e menu delle sessioni

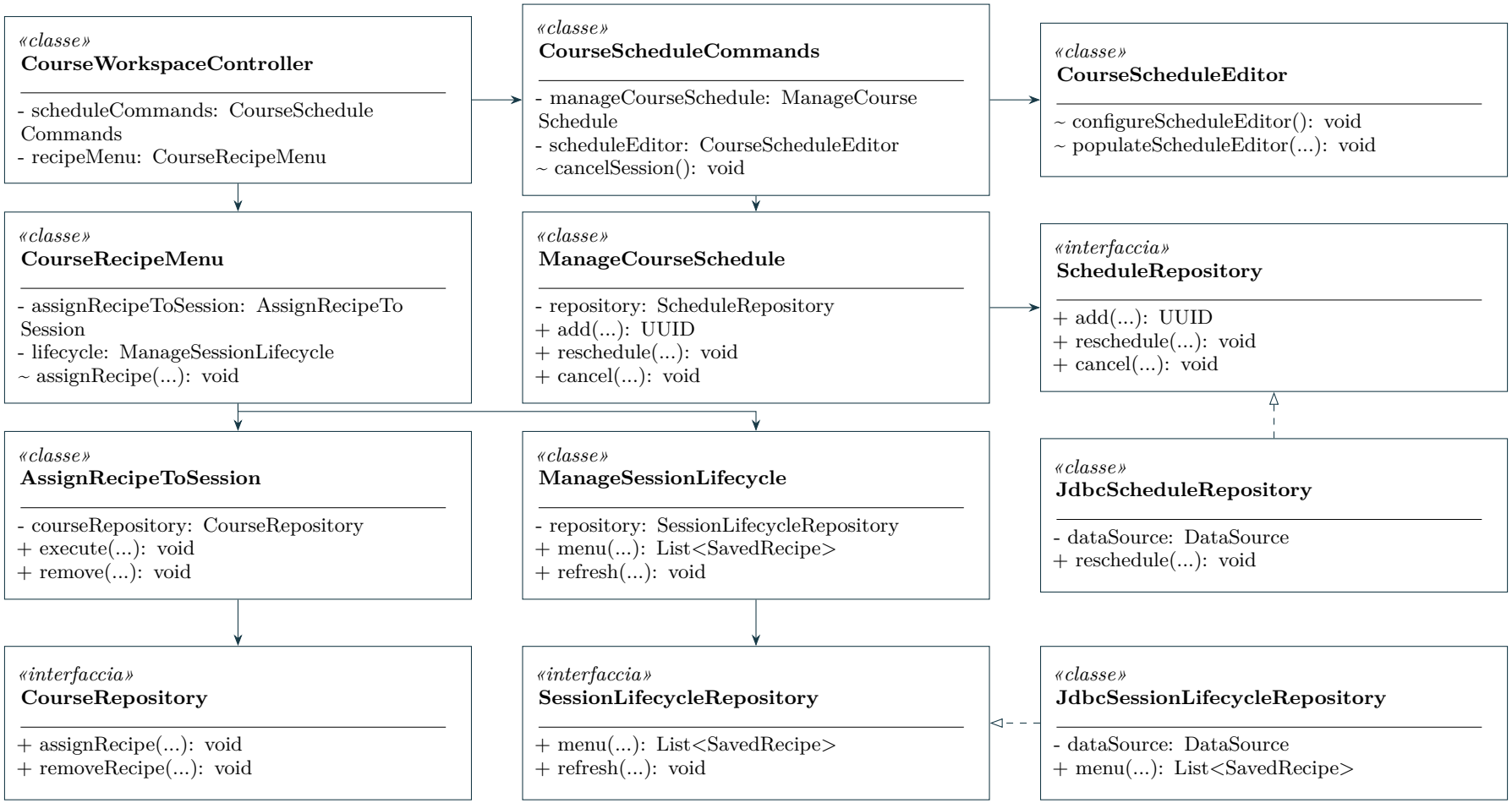


Figura 18: Calendario dei corsi e menu delle sessioni

I comandi del calendario includono la bozza della comunicazione. L'associazione di una ricetta e la lettura o l'aggiornamento delle copie del menu seguono contratti distinti. CourseRepository e JdbcCourseRepository sono collegati nella tavola sulla creazione del corso.

5.8 Consultazione e iscrizioni del Partecipante

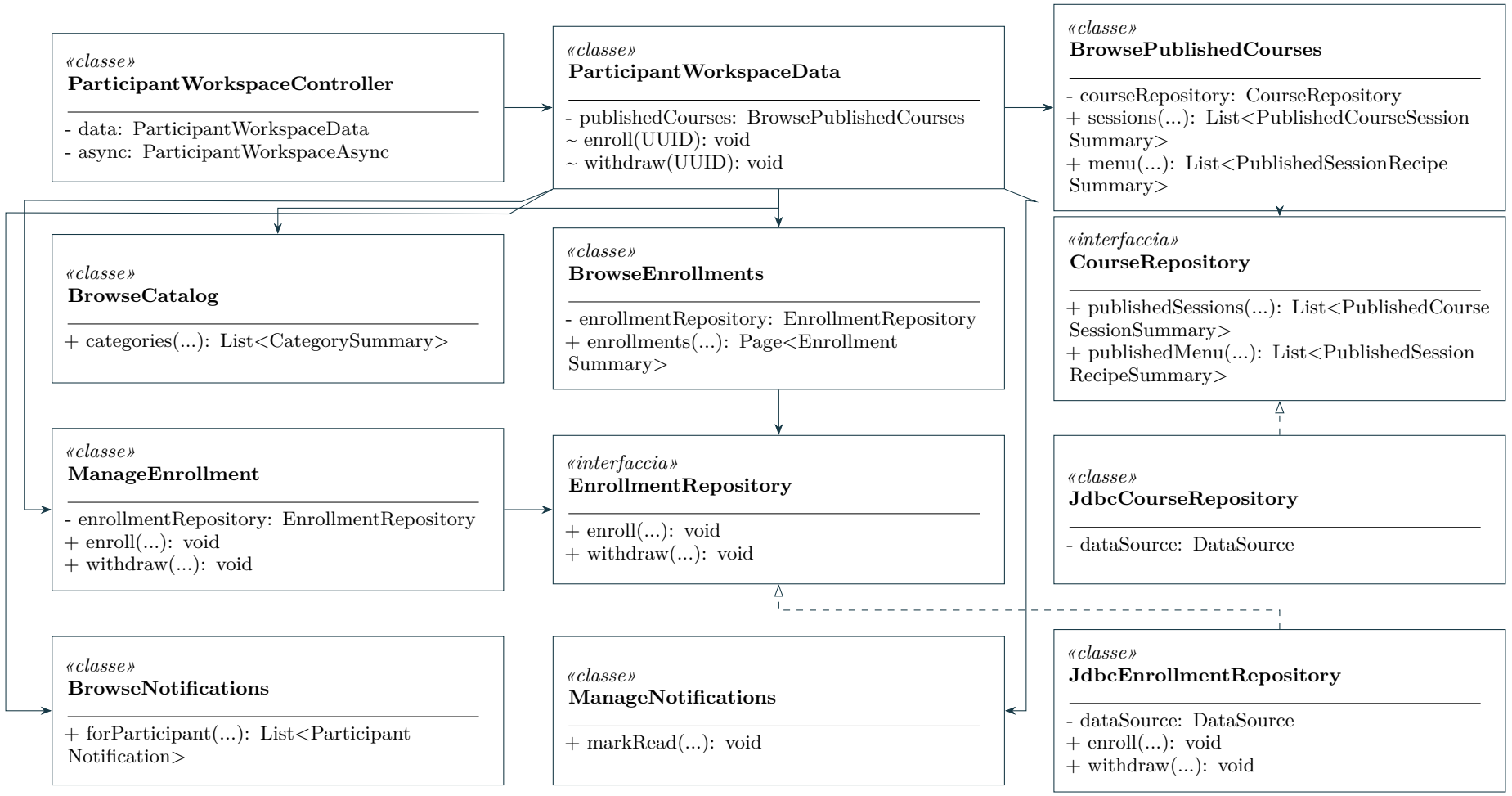


Figura 19: Consultazione e iscrizioni del Partecipante

ParticipantWorkspaceData unisce letture e operazioni della stessa area. La consultazione usa modelli di lettura; EnrollmentRepository resta separato da CourseRepository. Le adesioni sono dettagliate nella tavola dedicata, mentre BrowseNotifications e ManageNotifications sono ripresi nella tavola delle comunicazioni.

5.9 Comunicazioni e report mensile

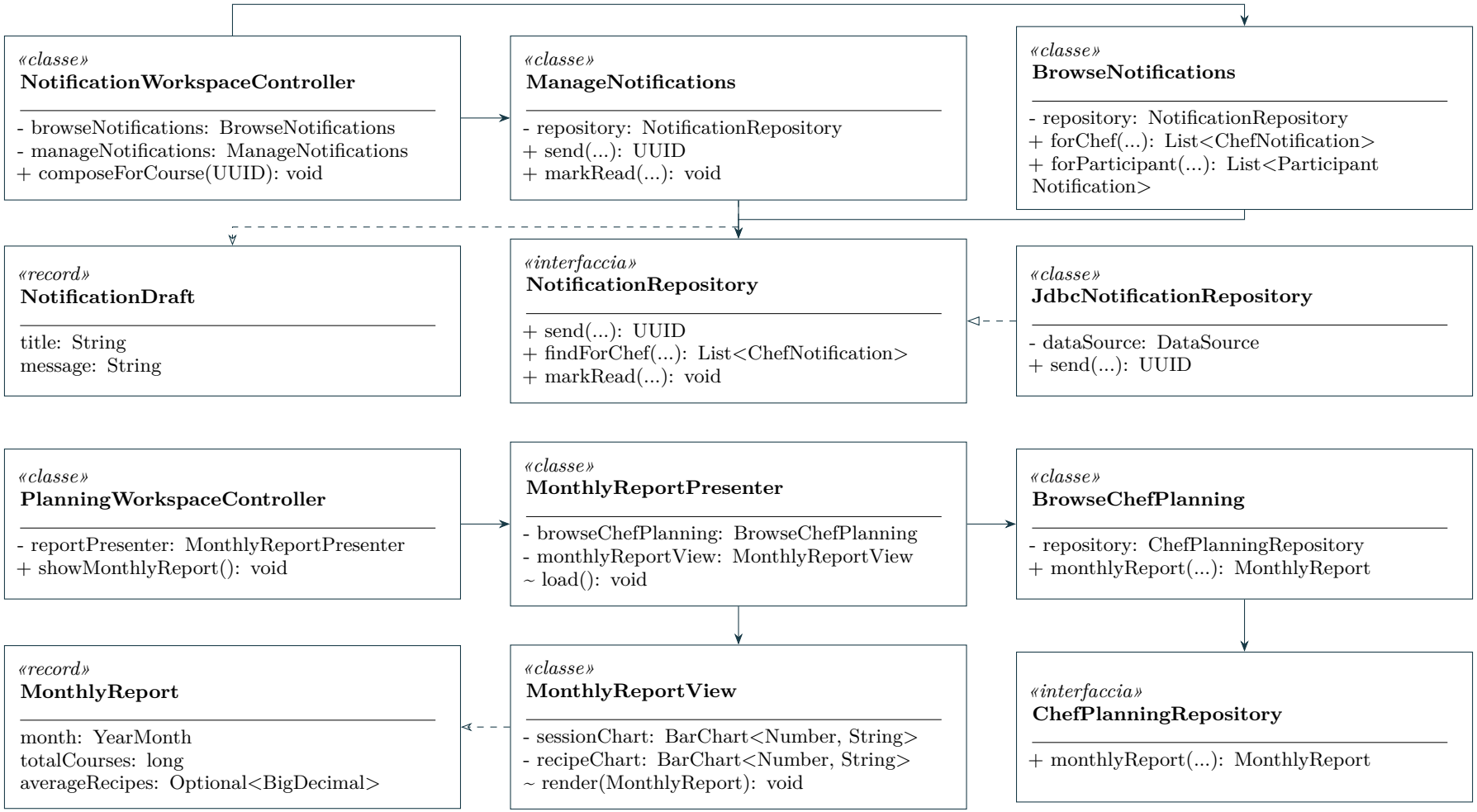


Figura 20: Comunicazioni e report mensile

Le comunicazioni vengono registrate e consultate nell'applicazione. Il report usa il medesimo BrowseChefPlanning della pianificazione, con ChefPlanningRepository e la sua implementazione JDBC già collegati nella tavola di lettura della pianificazione. MonthlyReportView costruisce i grafici a partire dal risultato immutabile.

5.10 Consultazione dei corsi Chef e agenda

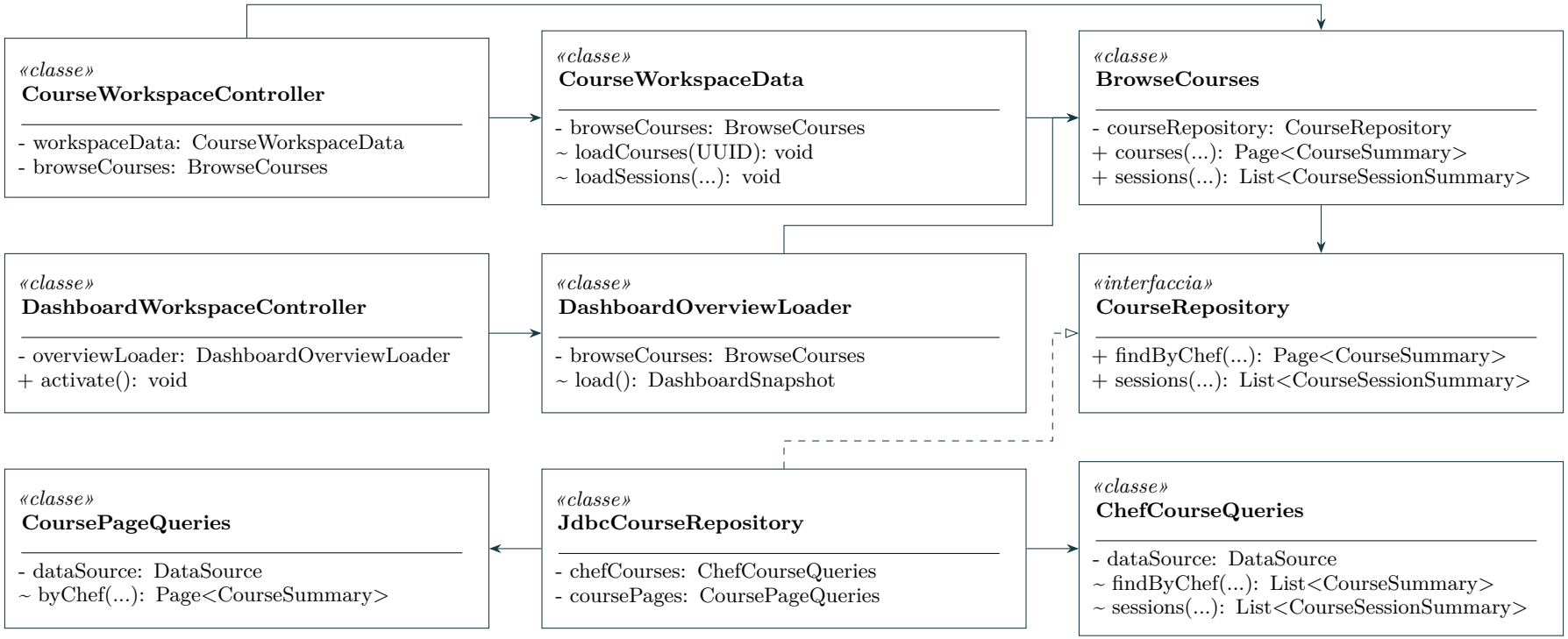


Figura 21: Consultazione dei corsi Chef e agenda

L'elenco filtrato e l'agenda riutilizzano BrowseCourses. Il repository delega le letture paginabili a CoursePageQueries e le letture delle sessioni a ChefCourseQueries: entrambe interrogano le funzioni SQL e restituiscono modelli di lettura, senza ricostruire una classe Corso persistente.

6 Comportamento di due funzionalità

6.1 Creazione e pubblicazione del corso

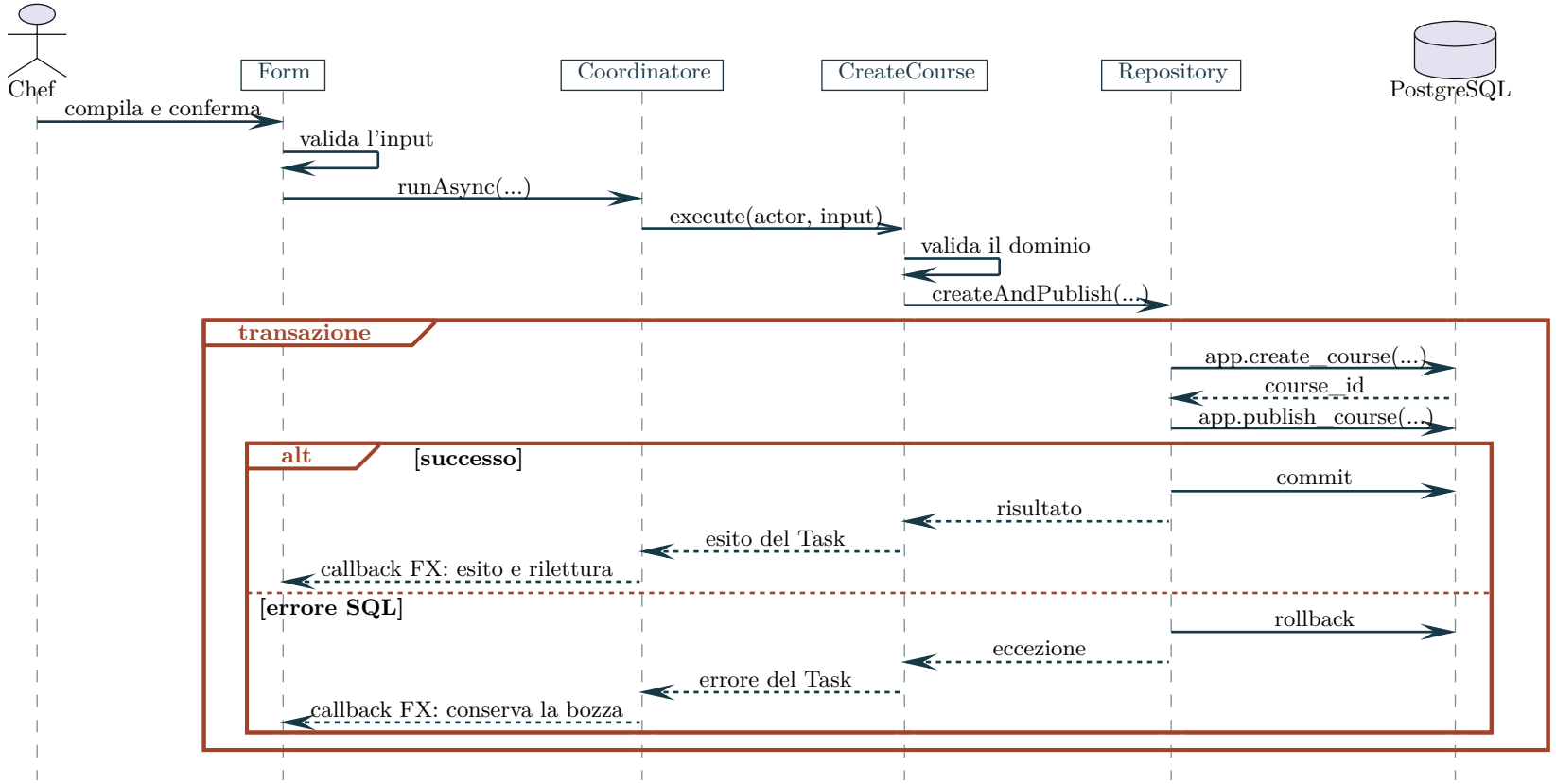


Figura 22: Creazione e pubblicazione del corso

Nella figura 22, Form indica CourseCreationForm, Coordinatore indica CourseWorkspaceCoordinator con FxAsyncExecutor e Task, Repository indica JdbcCourseRepository. Il ramo di rollback è alternativo a quello di successo. Un input non valido termina prima della chiamata al repository. Il form chiama CourseWorkspaceCoordinator attraverso CourseWorkspaceActions; il coordinatore usa FxAsyncExecutor. La bozza di input non equivale a un corso DRAFT già salvato.

6.2 Conferma di adesione alla pratica

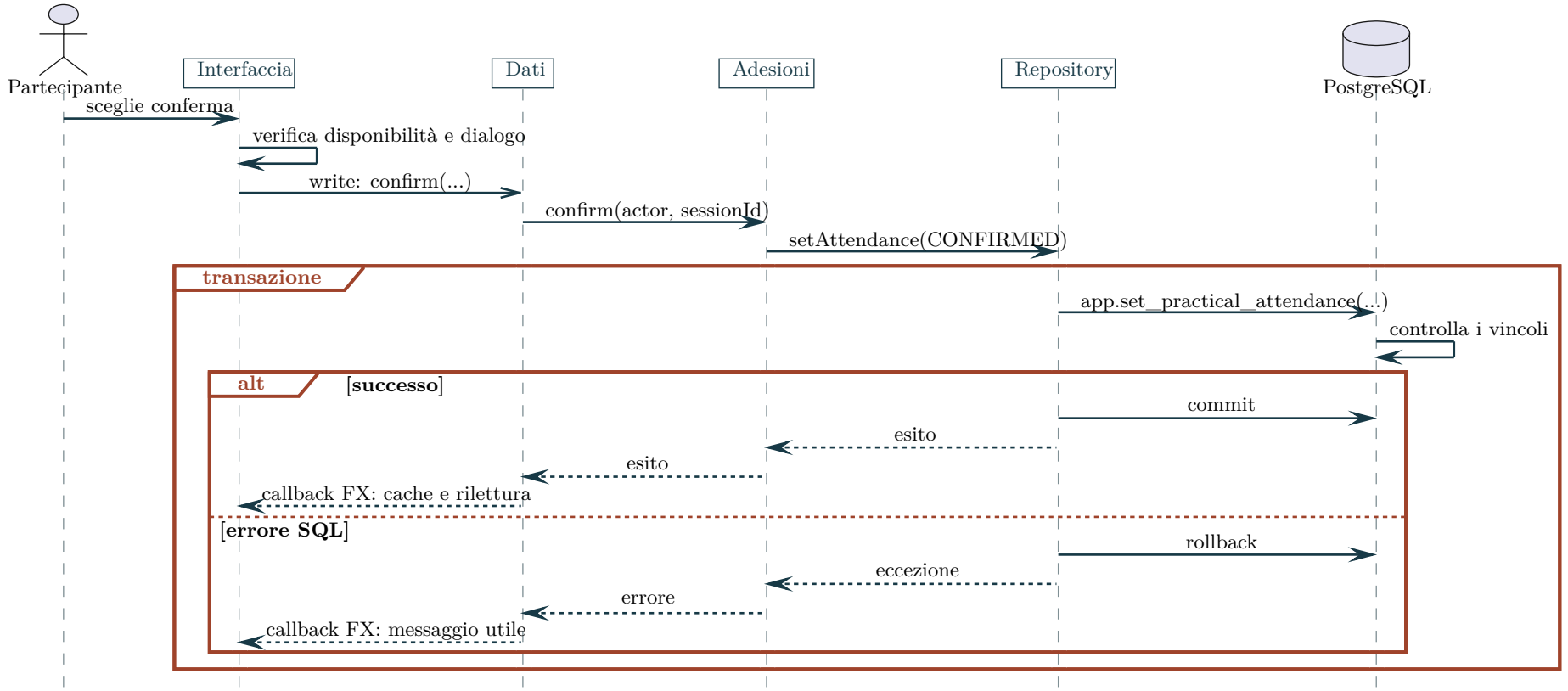


Figura 23: Conferma di adesione alla pratica

Nella figura 23, Interfaccia raggruppa ParticipantWorkspaceController e ParticipantWorkspaceAsync; Dati indica ParticipantWorkspaceData, Adesioni indica ManagePracticalAttendance e Repository indica JdbcEnrollmentRepository. Il raggruppamento mantiene visibile il passaggio asincrono senza introdurre una nuova classe. Se il partecipante annulla il dialogo, il controller termina senza chiamare il caso d'uso. La risposta di lettura obsoleta viene scartata tramite versione del contesto. Una scrittura riuscita e una successiva rilettura fallita sono esiti distinti: il secondo non prova che il salvataggio sia fallito. Le scritture non vengono ripetute automaticamente dall'helper asincrono.

7 Realizzazione della persistenza

Lo schema app utilizza ruoli distinti. Il migratore può modificare lo schema; il ruolo applicativo dispone dei privilegi necessari alle operazioni esposte. La migrazione iniziale V001 definisce tabelle, vincoli, indici, funzioni, trigger e privilegi; V002 aggiorna il controllo temporale dello spostamento delle sessioni. Flyway applica le migrazioni in ordine e controlla versione e checksum di ciascun file. Una correzione viene aggiunta senza modificare la migrazione già applicata, così anche i database esistenti ricevono lo stesso aggiornamento. I dati sintetici della demo e delle prove sono separati dalla struttura.

Il catalogo effettivo comprende 81 routine, tutte funzioni PostgreSQL, e 11 trigger applicativi. Non sono presenti procedure con prokind=p nel catalogo corrente: le operazioni sono implementate come funzioni SQL/PLpgSQL e trigger. Molte funzioni autorizzate usano SECURITY DEFINER con search_path controllato. Il JDBC usa PreparedStatement e parametri per gli input.

Le operazioni di dominio passano normalmente attraverso l'API SQL. Alcune letture tecniche autorizzate, come quelle di schema_metadata, accedono direttamente alle tabelle di supporto. Il percorso di accesso dipende quindi dalla natura dell'operazione.

7.1 Struttura SQL delle tabelle: casi significativi

La definizione completa dello schema è nel file del progetto `src/main/resources/db/migration/V001__create_initial_schema.sql`. Il file contiene le 22 tabelle, i vincoli introdotti con ALTER TABLE, gli indici, le funzioni, i trigger e i privilegi. Lo schema logico e il dizionario degli attributi sono descritti nella sezione 4.8; qui sono approfondite tre scelte che richiedono una spiegazione ulteriore.

7.1.1 Sessioni online e pratiche

Una sola tabella rappresenta le due modalità perché data, durata e collegamento al corso sono comuni. I CHECK rendono invece obbligatori i dati specifici: una sessione online richiede un link HTTPS e non usa sede o capienza; una pratica richiede sede, capienza positiva e chiusura delle adesioni precedente all'inizio. Il codice 1 mostra questi vincoli nella definizione fisica. Le chiavi, aggiunte separatamente nella migrazione, conservano il collegamento al corso e l'unicità del numero di sessione al suo interno.

```
CREATE TABLE app.course_session (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    course_id uuid NOT NULL,
    sequence_number integer NOT NULL,
    scheduled_at timestamp without time zone NOT NULL,
    mode character varying(20) NOT NULL,
    status character varying(20) DEFAULT 'SCHEDULED'::character varying NOT NULL,
    duration_minutes integer DEFAULT 120 NOT NULL,
    online_url text,
    venue character varying(180),
    capacity integer,
    attendance_closes_at timestamp without time zone,
    completed_at timestamp with time zone,
    CONSTRAINT course_session_attendance_cutoff_valid CHECK (((((mode)::text = 'ONLINE'::text)
    AND (attendance_closes_at IS NULL)) OR (((mode)::text = 'PRACTICAL'::text) AND
    (attendance_closes_at IS NOT NULL) AND (attendance_closes_at < scheduled_at)))),
    CONSTRAINT course_session_duration_valid CHECK (((duration_minutes >= 1) AND
    (duration_minutes <= 720))),
    CONSTRAINT course_session_mode_check CHECK (((mode)::text = ANY ((ARRAY['ONLINE'::character
    varying, 'PRACTICAL'::character varying])::text[]))),
    CONSTRAINT course_session_mode_fields_valid CHECK (((((mode)::text = 'ONLINE'::text) AND
    (online_url IS NOT NULL) AND (online_url ~*
    '^https://[[:alnum:]][[:alnum:].-]*(:[0-9]+)?([/?#][^[:space:]]*)?$'::text) AND (venue IS
    NULL) AND (capacity IS NULL)) OR (((mode)::text = 'PRACTICAL'::text) AND (online_url IS NULL)
    AND (venue IS NOT NULL) AND ((venue)::text = btrim((venue)::text)) AND (btrim((venue)::text)
    <> ''::text) AND (capacity IS NOT NULL) AND (capacity > 0)))),
    CONSTRAINT course_session_sequence_number_check CHECK ((sequence_number > 0)),
    CONSTRAINT course_session_status_check CHECK (((status)::text = ANY
    ((ARRAY['SCHEDULED'::character varying, 'COMPLETED'::character varying,
    'CANCELLED'::character varying])::text[])))
);
```

Codice 1: Dati comuni e vincoli delle due modalità di sessione

7.1.2 Dosi esatte per una persona

La tabella associativa `recipe_ingredient` conserva la quantità della coppia ricetta/ingrediente. La chiave composta impedisce una dose duplicata per lo stesso ingrediente. La colonna NUMERIC conserva il valore inserito; il controllo nel codice 2 ammette quantità positive inferiori a 10^{12} e con massimo tre decimali. Il confronto con round verifica la precisione e rifiuta il valore non valido, senza correggerlo mediante arrotondamento.

```
CREATE TABLE app.recipe_ingredient (
    recipe_id uuid NOT NULL,
    ingredient_id uuid NOT NULL,
    quantity numeric NOT NULL,
    CONSTRAINT recipe_ingredient_quantity_check CHECK ((quantity > (0)::numeric)),
    CONSTRAINT recipe_quantity_precision CHECK (((quantity > (0)::numeric) AND (quantity <
    ('1000000000000'::bigint)::numeric) AND (quantity = round(quantity, 3))))
);
```

Codice 2: Associazione tra ricetta e ingrediente con controllo della precisione

7.1.3 Copie storiche del menu

La copia della ricetta conserva insieme nome, descrizione, unità e dosi. JSONB permette di acquisire questo insieme senza collegarlo ai valori successivi del ricettario. È una ridondanza intenzionale: il menu della sessione deve restare interpretabile anche dopo la modifica di una ricetta. Il codice 3

mostra struttura e provenienza della copia; la FK composta, definita nel file completo, la collega alla coppia di `session_recipe`.

La provenienza CAPTURED distingue una copia acquisita da una ricostruzione LEGACY_RECONSTRUCTED. Una ricostruzione non prova quali fossero le dosi originarie. Prima della chiusura delle adesioni è consentito l'aggiornamento esplicito dal ricettario; le normali modifiche del ricettario non riscrivono automaticamente le copie.

```
CREATE TABLE app.session_menu_snapshot (
    session_id uuid NOT NULL,
    recipe_id uuid NOT NULL,
    recipe_name text NOT NULL,
    description text NOT NULL,
    ingredients jsonb NOT NULL,
    captured_at timestamp with time zone DEFAULT clock_timestamp() NOT NULL,
    provenance text NOT NULL,
    CONSTRAINT session_menu_snapshot_provenance_check CHECK ((provenance = ANY
    (ARRAY['CAPTURED'::text, 'LEGACY_RECONSTRUCTED'::text])))
);
```

Codice 3: Copia della ricetta associata alla sessione

7.2 Trigger e funzioni SQL: operazioni significative

PK, FK, UNIQUE e CHECK proteggono identità, riferimenti e condizioni sulla singola riga. Le regole che coinvolgono più righe richiedono invece controlli coordinati. Le operazioni applicative passano attraverso funzioni che verificano anche token, ruolo e proprietà dei dati; i trigger mantengono i vincoli sulle modifiche delle tabelle.

La realizzazione corrente comprende funzioni SQL/PLpgSQL e trigger, senza procedure SQL. Le dichiarazioni e i corpi completi della struttura iniziale sono disponibili in `src/main/resources/db/migration/V001__create_initial_schema.sql`; la correzione successiva è in `src/main/resources/db/migration/V002__protect_past_session_schedules.sql`. Entrambi i file appartengono alla directory delle migrazioni e definiscono insieme il comportamento corrente. Sono riportati soltanto i casi in cui la copia storica, la concorrenza o il calcolo dei risultati meritano attenzione.

La funzione `reschedule_course_session` blocca una modifica quando l'inizio originale è già stato raggiunto, confrontando l'appuntamento con l'ora dell'operazione nel fuso Europe/Rome. V002 applica questo controllo anche quando il parametro della sede o del link è assente: conservarne il valore non deve consentire di spostare una sessione già iniziata. La funzione esistente viene sostituita con `CREATE OR REPLACE`, mantenendo firma, autorizzazioni e transazione; tabelle, vincoli e numero delle routine restano invariati.

7.2.1 Acquisizione automatica della ricetta

Il trigger del codice 4 interviene dopo l'associazione della ricetta alla pratica. La funzione del codice 5 acquisisce nome e descrizione, costruisce l'elenco JSONB degli ingredienti e registra la provenienza. L'ordinamento per identificatore rende stabile la rappresentazione. La copia e l'associazione appartengono alla stessa transazione: un errore nell'acquisizione impedisce di salvare un'associazione ordinaria priva della propria copia.

```
CREATE TRIGGER capture_session_recipe AFTER INSERT ON app.session_recipe FOR EACH ROW EXECUTE
    FUNCTION app.capture_session_recipe();
```

Codice 4: Trigger di acquisizione della copia dopo l'associazione

```
CREATE FUNCTION app.capture_session_recipe() RETURNS trigger
    LANGUAGE plpgsql
    SET search_path TO 'pg_catalog', 'app', 'pg_temp'
    AS $$
BEGIN
    INSERT INTO app.session_menu_snapshot(session_id, recipe_id, recipe_name, description,
    ingredients, provenance)
    SELECT NEW.session_id, r.id, r.name, r.description,
        coalesce((SELECT jsonb_agg(jsonb_build_object('ingredient_id', i.id,
            'ingredient_name', i.name, 'unit', i.unit, 'quantity', ri.quantity) ORDER BY i.id)
            FROM app.recipe_ingredient ri JOIN app.ingredient i ON i.id=ri.ingredient_id
            WHERE ri.recipe_id=r.id), '[]'::jsonb), 'CAPTURED'
    FROM app.recipe r WHERE r.id=NEW.recipe_id;
    RETURN NEW;
END; $$;
```

Codice 5: Acquisizione della copia del menu

7.2.2 Capienza e validità delle adesioni

Due conferme simultanee non devono occupare lo stesso ultimo posto. La funzione `enforce_practical_attendance`, richiamata dal trigger del codice 6, blocca le righe di iscrizione e sessione con FOR UPDATE prima di verificare coerenza e capienza. Il conteggio considera soltanto iscrizioni attive e risposte del periodo corrente: una reiscrizione non riattiva una conferma precedente.

Il codice 7 è un estratto del ramo per le conferme, successivo all'acquisizione dei blocchi. La funzione completa controlla anche che la pratica appartenga al corso dell'iscrizione e aggiorna il momento della risposta. La funzione di accesso `set_practical_attendance` verifica inoltre scadenza delle adesioni e presenza del menu; questi controlli non sono attribuiti al solo trigger. Entrambe le funzioni sono nel file SQL completo.

```
CREATE TRIGGER practical_attendance_guard BEFORE INSERT OR UPDATE OF enrollment_id, session_id,
    status ON app.practical_attendance FOR EACH ROW EXECUTE FUNCTION
    app.enforce_practical_attendance();
```

Codice 6: Trigger di protezione delle adesioni alla pratica

```
    IF NEW.status = 'CONFIRMED' THEN
        IF enrollment_status <> 'ACTIVE'
                OR session_status <> 'SCHEDULED'
                OR session_time <= (current_timestamp AT TIME ZONE 'Europe/Rome') THEN
            RAISE EXCEPTION 'Attendance requires an active enrollment and a future session'
                USING ERRCODE = '23514';
        END IF;

        SELECT count(*)::INTEGER
          INTO confirmed_count
          FROM app.practical_attendance
          JOIN app.enrollment current_enrollment ON current_enrollment.id =
    practical_attendance.enrollment_id
         WHERE practical_attendance.session_id = NEW.session_id
           AND practical_attendance.status = 'CONFIRMED'
           AND current_enrollment.status = 'ACTIVE'
           AND practical_attendance.updated_at >= current_enrollment.enrolled_at
           AND practical_attendance.enrollment_id <> NEW.enrollment_id;
        IF confirmed_count >= session_capacity THEN
            RAISE EXCEPTION 'Practical session capacity has been reached'
                USING ERRCODE = '23514';
        END IF;
    END IF;
```

Codice 7: Estratto: conferme valide e controllo della capienza dopo il blocco delle righe

7.2.3 Appuntamenti dello stesso chef

Un lock consultivo di transazione, calcolato sullo chef, coordina le modifiche concorrenti al suo calendario. La funzione del codice 8 confronta gli intervalli delle sessioni programmate ed esclude la sessione in modifica. Le disuguaglianze strette consentono appuntamenti consecutivi: una sessione può iniziare esattamente quando termina l'altra. Il trigger `chef_session_overlap_guard`, definito nel file completo, attiva il controllo all'inserimento e alla modifica dei dati dell'appuntamento.

```
CREATE FUNCTION app.prevent_chef_session_overlap() RETURNS trigger
    LANGUAGE plpgsql SECURITY DEFINER
    SET search_path TO 'pg_catalog', 'app', 'pg_temp'
    AS $$
DECLARE owner_id UUID;
BEGIN
    IF NEW.status <> 'SCHEDULED' THEN
        RETURN NEW;
    END IF;

    SELECT chef_profile_id INTO owner_id FROM app.course WHERE id = NEW.course_id;
    IF owner_id IS NULL THEN
        RAISE EXCEPTION 'Course owner is unavailable' USING ERRCODE = '23503';
    END IF;

    PERFORM pg_advisory_xact_lock(hashtextextended(owner_id::TEXT, 0));
    IF EXISTS (
        SELECT 1
          FROM app.course_session existing_session
          JOIN app.course existing_course ON existing_course.id = existing_session.course_id
         WHERE existing_course.chef_profile_id = owner_id
           AND existing_session.status = 'SCHEDULED'
           AND existing_session.id IS DISTINCT FROM NEW.id
           AND existing_session.scheduled_at
                 < NEW.scheduled_at + make_interval(mins => NEW.duration_minutes)
           AND NEW.scheduled_at
                 < existing_session.scheduled_at + make_interval(mins =>
    existing_session.duration_minutes)
    ) THEN
        RAISE EXCEPTION 'Chef already has another session in this time interval'
            USING ERRCODE = '23P01';
    END IF;
    RETURN NEW;
END;
$$;
```

Codice 8: Blocco per chef e verifica degli intervalli del calendario

7.2.4 Fabbisogni dalle copie e dalle risposte correnti

La query del codice 9 separa le risposte degli iscritti dalle dosi del menu. Il collegamento temporale esclude le risposte anteriori all'iscrizione corrente. JSONB viene trasformato in righe; la somma per identificatore di ingrediente è poi moltiplicata per le conferme valide. Le quantità e le unità provengono dalla copia, mentre il nome mostrato è quello corrente del catalogo.

`preliminary_session_preparation` è una funzione interna di calcolo. La funzione applicativa `session_preparation`, presente nel file completo, verifica l'identità dello chef e la proprietà della pratica prima di richiamarla. La separazione permette di riusare il calcolo senza rendere pubblica una lettura priva di autorizzazione.

Per un ingrediente i della pratica s, il fabbisogno è $Q_{s,i} = n_s \sum_{r \in M_s} q_{r,i}$: n_s indica le conferme valide, M_s il menu copiato e $q_{r,i}$ la dose per una persona. Con 6 conferme e dosi di 100 g e 50 g dello stesso ingrediente, il totale è 900 g.

```
CREATE FUNCTION app.preliminary_session_preparation(p_session uuid) RETURNS
    TABLE(confirmed_participants integer, pending_participants integer, declined_participants
    integer, ingredient_margin numeric, recipe_id uuid, recipe_name text, base_servings integer,
    portions_per_participant numeric, ingredient_id uuid, ingredient_name text, unit text,
    contribution_quantity numeric, required_quantity numeric)
    LANGUAGE sql STABLE
    SET search_path TO 'pg_catalog', 'app', 'pg_temp'
    AS $$
    WITH responses AS (
        SELECT count(*) FILTER(WHERE pa.status='CONFIRMED')::INTEGER confirmed,
            count(*) FILTER(WHERE pa.status IS NULL)::INTEGER pending,
            count(*) FILTER(WHERE pa.status='DECLINED')::INTEGER declined
        FROM app.enrollment e JOIN app.course_session s ON s.course_id=e.course_id
        LEFT JOIN app.practical_attendance pa ON pa.enrollment_id=e.id AND pa.session_id=s.id AND
    pa.updated_at>=e.enrolled_at
        WHERE s.id=p_session AND e.status='ACTIVE'
    ), doses AS (
        SELECT m.recipe_id,m.recipe_name,i.* FROM app.session_menu_snapshot m
        CROSS JOIN LATERAL jsonb_to_recordset(m.ingredients)
            AS i(ingredient_id UUID,ingredient_name TEXT,unit TEXT,quantity NUMERIC)
        WHERE m.session_id=p_session
    )
    SELECT r.confirmed,r.pending,r.declined,0::NUMERIC,d.recipe_id,d.recipe_name,1,1::NUMERIC,
        d.ingredient_id,i.name::TEXT,d.unit,d.quantity*r.confirmed,
        sum(d.quantity) OVER(PARTITION BY d.ingredient_id)*r.confirmed
    FROM responses r LEFT JOIN doses d ON TRUE
    LEFT JOIN app.ingredient i ON i.id=d.ingredient_id;
$$;
```

Codice 9: Calcolo dei fabbisogni per ingrediente dalle copie del menu

7.2.5 Svolgimento derivato e report mensile

Lo svolgimento è ricavato dalla fine prevista e mantiene la precedenza dell'annullamento. Il codice 10 rende il calcolo indipendente dall'apertura dell'applicazione o da un processo pianificato. L'orario di confronto è interpretato nel fuso Europe/Rome dalle funzioni chiamanti.

```
CREATE FUNCTION app.effective_session_status(p_status text, p_scheduled_at timestamp without time
    zone, p_duration_minutes integer, p_as_of timestamp without time zone) RETURNS text
    LANGUAGE sql IMMUTABLE STRICT
    SET search_path TO 'pg_catalog', 'app', 'pg_temp'
    AS $$
    SELECT CASE WHEN p_status='CANCELLED' THEN 'CANCELLED'
        WHEN p_scheduled_at+make_interval(mins=>p_duration_minutes)<=p_as_of THEN 'COMPLETED'
        ELSE 'SCHEDULED' END;
$$;
```

Codice 10: Stato effettivo derivato dalla fine prevista della sessione

Il codice 11 aggrega soltanto le sessioni svolte. Il numero di corsi è distinto; media, minimo e massimo delle ricette riguardano le sole pratiche. La funzione `chef_monthly_report_details`, disponibile nel file completo, controlla ruolo e mese richiesto, attribuisce le sessioni al mese della data programmata e conta le copie delle ricette associate. Si evita così di duplicare nel riepilogo la regola temporale e i controlli di accesso.

```
CREATE FUNCTION app.chef_monthly_report(p_session_token uuid, p_year integer, p_month integer)
    RETURNS TABLE(total_courses bigint, online_sessions bigint, practical_sessions bigint,
    average_recipes numeric, minimum_recipes integer, maximum_recipes integer)
    LANGUAGE sql STABLE SECURITY DEFINER
    SET search_path TO 'pg_catalog', 'app', 'pg_temp'
    AS $$
    SELECT count(DISTINCT s.course_id),count(*) FILTER(WHERE d.mode='ONLINE'),count(*)
    FILTER(WHERE d.mode='PRACTICAL'),
        avg(d.recipes) FILTER(WHERE d.mode='PRACTICAL'),min(d.recipes) FILTER(WHERE
    d.mode='PRACTICAL'),max(d.recipes) FILTER(WHERE d.mode='PRACTICAL')
    FROM app.chef_monthly_report_details(p_session_token,p_year,p_month) d JOIN
    app.course_session s ON s.id=d.session_id
    WHERE d.state='COMPLETED';
$$;
```

Codice 11: Indicatori del report mensile dello chef

7.3 Autenticazione e gestione delle connessioni

Le password sono protette con PBKDF2 e sale casuale; `auth_session` conserva l'hash del token di accesso. Le routine verificano scadenza, account attivo e ruolo, oltre alla proprietà o visibilità dei dati richiesti. Questi controlli proteggono le operazioni anche oltre la validazione della GUI.

Migrazione e accesso applicativo usano credenziali separate per limitare i privilegi del client. Le query JDBC sono parametrizzate; le operazioni lente vengono eseguite fuori dal thread JavaFX e gli errori sono tradotti in messaggi comprensibili.